

《面向对象程序设计实践》

课程论文

题目：	题目 2 思维导图绘制工具
专业：	软件工程
班级：	24 级 5 班
小组成员 1：	202425220507 陈宗波
2：	202425220505 陈宇冲
3：	202425220523 温文辉
指导老师：	彭红星
提交时间：	2026 年 05 月 16 日

目录

1 系统分析	4
1.1 问题描述	4
1.2 系统功能分析	4
1.2.1 新建思维导图功能	4
1.2.2 保存思维导图功能	4
1.2.3 打开思维导图功能	4
1.2.4 添加节点功能	4
1.2.5 删除节点功能	5
1.2.6 重命名节点功能	5
1.2.7 节点样式设置功能	5
1.2.8 导图布局设置功能	5
1.2.9 图片导出功能	5
1.2.10 结构树同步显示功能	5
1.2.11 撤销操作功能	6
1.2.12 画布缩放与中心定位功能	6
1.3 开发平台及工具介绍	6
1.3.1 开发语言与图形界面技术	6
1.3.2 开发工具	6
1.3.3 运行环境	6
2 系统设计	6
2.1 系统总体结构设计	6
2.1.1 系统模块划分	6
2.1.2 各模块功能说明	7
2.2 系统各个类及类之间关系设计	8
2.2.1 主要类的职责说明	8
2.2.2 类之间的关系设计	9
2.3 数据存储设计	10
2.3.1 思维导图文件结构设计	10
2.3.2 节点数据结构设计	10
2.3.3 节点样式数据设计	11
2.3.4 布局信息存储设计	12
2.3.5 节点位置偏移信息存储设计	12
2.4 界面设计	12
2.4.1 主界面总体设计	12
2.4.2 工具栏设计	13
2.4.3 绘图区设计	13
2.4.4 结构展示区设计	14
2.4.5 状态栏设计	14
2.4.6 右键菜单与属性设置界面设计	14
2.5 关键设计说明	14
2.5.1 节点选中机制设计	14
2.5.2 布局切换设计	15
2.5.3 界面同步设计	15

2.5.4 撤销机制设计	15
2.5.5 画布缩放与中心定位设计	16
2.5.6 文件覆盖保护设计	16
3 系统实现	16
3.1 界面交互模块实现	16
3.2 布局计算与绘制模块实现	18
3.3 文件管理模块实现	21
3.4 撤销与状态管理模块实现	23
3.5 缩放与图片导出模块实现	25
4 系统测试	27
4.1 测试步骤	27
4.2 模块测试	27
4.2.1 文件保存与读取模块测试 (JUnit)	27
4.2.2 布局计算模块测试 (JUnit)	28
4.2.3 节点样式持久化模块测试 (JUnit)	28
4.2.4 节点编辑功能测试 (手工测试)	29
4.2.5 中心节点删除限制测试 (手工测试)	29
4.2.6 结构树同步显示测试 (手工测试)	30
4.2.7 撤销功能测试 (手工测试)	30
4.2.8 画布缩放与中心定位测试 (手工测试)	30
4.2.9 图片导出模块测试 (手工测试)	31
4.3 系统测试	31
4.4 测试结论	32
5 系统运行界面	32
5.1 主界面截图	32
5.2 节点编辑功能截图	32
5.3 节点样式设置功能截图	33
5.4 布局切换效果截图	33
5.5 文件保存、打开与导出效果截图	35
5.6 画布缩放与中心定位效果截图	35
5.7 撤销截图	36
5.8 文件覆盖确认效果截图	37
6 总结	38

1 系统分析

1.1 问题描述

随着学习和工作中需要整理的信息越来越多，单纯使用普通文字记录内容时，容易出现层次不清、重点不突出的问题。思维导图可以用中心主题和分支节点的形式，把不同内容之间的层级关系展示出来，因此可以用来做知识整理和结构化表达。

本次课程设计选择的题目是“思维导图绘制工具”。该题目要求实现一个能够绘制思维导图的图形界面程序，使用户可以通过可视化方式创建和编辑导图。根据课程设计要求，本系统需要支持单一中心节点的思维导图，并能够完成导图的新建、保存、打开、节点编辑、布局设置以及图片导出等操作。

从程序实现角度来看，本系统不仅是简单地搭建一个窗口界面，还需要处理图形绘制、鼠标事件、树形数据结构、布局计算、文件读写、图片导出以及结构树同步显示等内容。

1.2 系统功能分析

1.2.1 新建思维导图功能

新建思维导图功能是本系统的基础功能。用户进入程序后，可以通过菜单栏或工具栏创建一张新的思维导图。

新建导图后，系统会生成一个中心节点，作为整张思维导图的起点。用户后续添加的其他节点都围绕中心节点展开，从而形成一个具有层级关系的树形结构。这个功能主要用于让用户从空白状态开始创建新的导图内容。

如果用户当前正在编辑的导图还没有保存，系统在新建前需要进行提示，避免用户误操作导致原有内容丢失。新建完成后，界面会重新显示新的中心节点，用户可以继续进行后续编辑。

1.2.2 保存思维导图功能

保存思维导图功能用于将当前正在编辑的导图保存到本地文件中。用户在完成导图编辑后，可以通过保存功能保留当前内容，方便以后继续打开和修改。

本系统保存的内容不仅包括节点文字，还包括节点之间的层级关系、导图标题、布局方式和节点样式等信息。这样用户下次打开文件时，系统能够恢复出之前保存的导图内容和显示效果。

本系统使用自定义的.dt 文件作为思维导图文件。第一次保存时，用户需要选择保存位置并输入文件名；后续再次保存时，可以直接保存到原文件中。这样既方便首次保存，也方便后续编辑过程中的快速保存。

为了避免误覆盖已有文件，如果用户保存时选择了已经存在的文件，系统需要给出确认提示。只有用户确认后，系统才会继续保存，从而减少误操作造成的数据丢失。

1.2.3 打开思维导图功能

打开思维导图功能用于读取之前保存过的.dt 文件。用户可以从本地选择已经保存的导图文件，系统读取后将导图重新显示在界面中。

打开文件后，系统需要恢复导图的中心节点、普通节点、节点层级关系、布局方式和节点样式等内容。这样用户可以在之前保存的基础上继续编辑，而不需要重新创建导图。

这个功能的作用是支持导图的持续编辑。对于课程笔记、项目分析或实验报告内容来说，用户往往不可能一次完成所有内容，因此打开已有文件继续修改是一个必要功能。

1.2.4 添加节点功能

添加节点功能用于扩展思维导图内容。用户可以通过添加节点功能，在已有节点的基础上继续补充新的内容。

本系统的节点添加包括添加子节点和添加兄弟节点两种方式。添加子节点表示在当前节

点下面增加一个下级内容；添加兄弟节点表示在当前节点的同一层级增加一个并列内容。

需要注意的是，中心节点是整张导图的唯一根节点，因此中心节点不能添加兄弟节点，只能添加子节点。

1.2.5 删除节点功能

删除节点功能用于移除导图中不需要的节点。用户选中某个节点后，可以执行删除操作，将该节点从当前思维导图中移除。

由于思维导图是树形结构，节点之间存在父子关系，所以删除一个节点时，它下面的子节点也应一起删除。如果只删除父节点而保留子节点，会导致导图结构不完整。

中心节点是整张思维导图的根节点，不允许删除。如果用户尝试删除中心节点，系统需要给出提示，防止用户破坏整张导图的基本结构。

删除功能可以帮助用户修改和调整导图内容，使用户能够及时去掉错误、不需要或重复的节点。

1.2.6 重命名节点功能

重命名节点功能用于修改节点显示的文字内容。在绘制思维导图的过程中，用户经常需要根据实际内容调整节点名称，因此系统需要提供节点重命名功能。

用户可以选中节点后进行重命名，也可以通过双击节点的方式快速修改节点文字。修改完成后，绘图区和结构树中的节点名称都需要同步更新。

如果用户修改的是中心节点，导图标题也应随之更新，使中心主题和当前导图名称保持一致。这样可以保证导图内容显示更加统一。

1.2.7 节点样式设置功能

节点样式设置功能用于调整节点的显示效果。用户可以通过设置节点样式，让不同类型或不同层级的节点在视觉上更加容易区分。

本系统支持设置节点填充颜色、边框颜色、文字颜色、字体大小、加粗效果和分支线颜色等内容。通过这些设置，用户可以突出重点节点，也可以让整张导图看起来更加清楚。

节点样式设置功能不仅提高了导图的美观性，也提高了内容表达的清晰度。对于节点较多的导图来说，适当的颜色和样式区分可以帮助用户更快理解导图结构。

1.2.8 导图布局设置功能

导图布局设置功能用于调整节点在绘图区中的排列方式。本系统支持自动布局、左侧布局和右侧布局三种模式。

自动布局适合大多数情况，系统会让中心节点的分支分布在左右两侧，使整体结构尽量保持平衡。左侧布局适合用户希望内容主要向左展开的情况；右侧布局适合用户希望内容主要向右展开的情况。

1.2.9 图片导出功能

图片导出功能用于将当前绘制好的思维导图保存为图片文件。本系统支持导出 PNG 和 JPG 两种格式。

.dt 文件主要用于后续继续编辑，而图片文件主要用于展示和使用。用户可以将导出的图片插入实验报告、课程作业、演示文稿或其他文档中。

图片导出功能可以提高导图成果的使用范围，使用户不仅能在本系统中查看导图，也可以在其他软件或文档中使用导图结果。

1.2.10 结构树同步显示功能

结构树同步显示功能用于在界面右侧显示当前思维导图的层级结构。绘图区主要以图形方式展示导图，而结构树则以目录形式展示节点之间的父子关系。

当用户添加节点、删除节点、重命名节点或打开导图文件时，结构树需要同步更新。这样可以保证结构树显示的内容和绘图区中的导图内容一致。

结构树可以帮助用户快速定位节点。当导图节点较多时，用户可以通过结构树更清楚地查看整体层级，也可以更方便地找到某个具体节点。

1.2.11 撤销操作功能

撤销操作功能用于提高用户编辑过程中的容错能力。用户在编辑思维导图时，可能会出现误删节点、误改节点名称、误改样式或切换了不合适的布局等情况，因此系统需要提供撤销功能。

通过撤销操作，用户可以将导图恢复到上一步编辑前的状态，减少误操作带来的影响。该功能能够让用户在编辑过程中更加放心，不必担心一次错误操作就破坏已有内容。

本系统支持通过快捷键 Ctrl+Z 执行撤销操作。撤销功能主要面向节点编辑、样式设置和布局调整等常见操作，使用户能够更灵活地修改导图内容。

1.2.12 画布缩放与中心定位功能

画布缩放与中心定位功能用于改善较大导图的查看体验。当导图节点较多、层级较深时，如果只能以固定比例查看，用户可能难以同时观察整体结构和局部细节。

本系统提供放大、缩小和恢复正常比例等功能。用户可以根据需要放大查看某个节点附近的细节，也可以缩小查看整张导图的大致结构。

中心定位功能用于帮助用户快速回到中心节点所在位置。当导图内容较多或者用户滚动到较远区域时，可以通过中心定位功能快速回到导图核心主题，方便继续编辑或查看。

这个功能主要提升系统的浏览和操作体验，特别适合节点数量较多的思维导图。

1.3 开发平台及工具介绍

1.3.1 开发语言与图形界面技术

本系统使用 Java 语言进行开发。Java 具有较好的面向对象特性，适合将思维导图中的文档、节点、布局、文件管理和界面控制等内容分别设计为不同的类。例如，节点类用于表示单个节点，文档类用于表示完整导图，布局类用于负责节点位置计算，文件管理类用于处理导图的保存与读取。

图形界面部分使用 Java Swing 实现。Swing 提供了窗口、菜单、按钮、工具栏、树形控件、滚动面板以及自定义绘图面板等组件，能够满足本系统桌面图形界面的开发需求。本系统中的主窗口、绘图区、结构树和状态栏等部分都基于 Swing 组件完成。

1.3.2 开发工具

本项目可以使用 IntelliJ IDEA 作为主要开发工具。该工具对 Java 项目的支持较为完善，能够方便地进行代码编写、运行、调试和错误检查。

本项目使用 Maven 进行项目构建和依赖管理。通过 pom.xml 文件可以统一管理项目结构、编译版本、测试依赖以及程序入口配置，使整个项目更加规范。为了提高程序稳定性，项目中还使用了 JUnit 5 对文件保存与读取、布局计算以及样式保存等功能进行了测试。

1.3.3 运行环境

本系统是一个本地运行的 Java 桌面程序，不依赖数据库，也不需要服务器环境。用户只需要在本地安装 Java 运行环境，即可启动程序并完成思维导图的新建、编辑、保存、打开和导出等操作。

本项目使用 JDK 17 进行开发和运行，能够在 Windows 环境下正常执行。

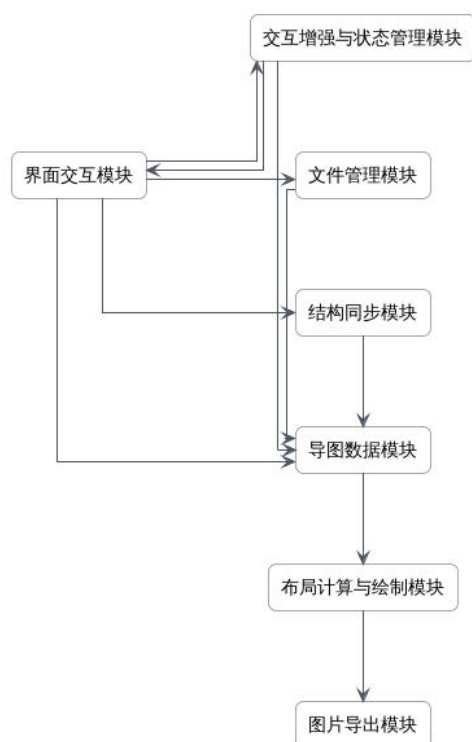
2 系统设计

2.1 系统总体结构设计

2.1.1 系统模块划分

系统划分为 7 个模块，分别是界面交互模块、导图数据模块、布局计算与绘制模块、文件管理模块、图片导出模块、结构同步模块以及交互增强与状态管理模块。

图2-1 系统模块划分图



2.1.2 各模块功能说明

系统各模块之间分工明确，并通过界面交互模块进行统一协调。用户在界面中执行操作后，系统会根据操作类型调用对应模块完成数据修改、界面刷新、文件读写、图片导出或状态更新等处理。

界面交互模块是用户与系统交互的入口，主要负责主窗口及各类界面元素的显示和事件响应。菜单栏、工具栏、绘图区、结构展示区、状态栏和节点右键属性面板都属于这一模块。用户进行新建、打开、保存、导出、添加节点、删除节点、重命名节点、设置样式、切换布局和缩放画布等操作时，首先由该模块接收，再分发给其他模块处理。

导图数据模块是系统的数据基础，主要负责保存思维导图的核心内容，包括导图标题、中心节点、节点文字、父子关系、节点样式、布局方式以及节点位置偏移量等。由于思维导图本身属于树形结构，因此该模块通过节点之间的父子关系组织整张导图。布局计算、结构同步、文件保存、文件打开和撤销恢复等功能都需要依赖导图数据模块。

布局计算与绘制模块主要负责节点位置计算和绘图区显示。系统根据当前节点结构和布局方式计算每个节点的显示区域，并在绘图区中绘制节点、连线、选中效果和样式效果。本系统支持自动布局、左侧布局和右侧布局三种方式。当用户添加或删除节点、切换布局、调整中心节点位置或缩放画布时，该模块会重新计算或刷新显示结果。

文件管理模块主要负责本地导图文件的保存与打开。本系统使用自定义的 .dt 文件保存思维导图内容，文件内部采用 XML 结构记录节点层级、布局方式、节点样式和位置偏移信息。用户保存文件时，该模块负责把当前导图数据写入本地文件；用户打开文件时，该模块负责解析文件内容，并恢复出对应的导图数据。

图片导出模块主要负责将当前思维导图导出为图片文件。本系统支持导出 PNG 和 JPG 两种图片格式。该模块会根据当前绘图区内容生成图片，使用户能够将导图结果用于课程报告、展示材料或其他文档中。它和文件管理模块的区别在于：文件管理模块保存的是可继续编辑的 .dt 文件，而图片导出模块输出的是用于展示的图片文件。

结构同步模块主要负责绘图区和结构树之间的联动。用户在绘图区添加、删除或重命名节点后，右侧结构树会同步更新；用户在结构树中选中某个节点时，绘图区也会同步选中对应节点。结构展示区还提供展开全部、定位中心和清空选择等辅助操作，方便用户在节点较多时快速浏览和定位。

交互增强与状态管理模块主要负责提高系统的易用性和容错能力。该模块包括撤销操作、未保存状态标记、画布缩放状态、中心定位、按钮可用状态控制、删除确认和导出覆盖确认等内容。通过这些设计，系统不仅能够完成基本的思维导图绘制功能，也能减少用户误操作带来的影响，并改善较大导图的查看和编辑体验。

总体来看，导图数据模块是系统的核心数据基础，界面交互模块负责协调用户操作，布局计算与绘制模块负责图形化显示，文件管理模块负责导图数据持久化，图片导出模块负责导图结果输出，结构同步模块负责保持不同视图的一致性，交互增强与状态管理模块负责提升系统稳定性和操作体验。各模块相互配合，共同完成思维导图的创建、编辑、保存、打开、导出和状态管理。

2.2 系统各个类及类之间关系设计

2.2.1 主要类的职责说明

本系统按照功能职责设计了多个类，每个类承担不同任务。主要类包括 MindMapApp、MainFrame、MindMapCanvas、MindMapDocument、MindMapNode、LayoutMode、MindMapLayoutEngine、LayoutSnapshot 和 MindMapFileManager。

各主要类职责如下表所示：

类名	所属包	主要职责
MindMapApp	com.course.mindmap	程序入口类，负责启动系统主窗口。
MainFrame	com.course.mindmap.ui	主界面类，负责菜单栏、工具栏、绘图区、结构树、状态栏、右键属性面板以及用户操作协调，同时负责撤销、未保存状态、缩放按钮、中心定位和导出覆盖确认等界面控制逻辑。
MindMapCanvas	com.course.mindmap.ui	绘图区类，负责节点绘制、连线绘制、节点选中、双击重命名、右键属性弹出、中心节点拖动、画布缩放、节点居中显示和图片导出绘制。
MindMapDocument	com.course.mindmap.model	思维导图文档类，保存导图标题、中心节点和布局方式，并提供导图复制与恢复功能，用于支持撤销操作。
MindMapNode	com.course.mindmap.model	思维导图节点类，保存节点文本、父子关系、节点样式以及位置偏移量，是构成导图树形结构的基础对象。
LayoutMode	com.course.mindmap.model	布局模式枚举类，表示自动布局、左侧布局和右侧布局。
MindMapLayoutEngine	com.course.mindmap.layout	布局计算类，负责根据节点结构、

		节点大小和布局模式计算各节点位置。
LayoutSnapshot	com.course.mindmap.layout	布局结果类，保存每个节点的显示区域和整张导图的边界信息，供绘图区绘制和导出时使用。
MindMapFileManager	com.course.mindmap.io	文件管理类，负责.dt 文件保存与读取，并处理导图标题、布局方式、节点结构、节点样式和位置偏移信息的持久化。

2.2.2 类之间的关系设计

本系统中各个类之间的关系比较明确。MindMapApp 是程序入口，负责创建 MainFrame，程序的主要界面和功能都在 MainFrame 中组织和协调。

MainFrame 中保存当前正在编辑的 MindMapDocument 对象，同时包含 MindMapCanvas 绘图区对象、右侧 JTree 结构树对象以及 MindMapFileManager 文件管理对象。用户在界面上执行操作时，MainFrame 根据操作内容修改 MindMapDocument 中的数据，然后通知 MindMapCanvas 刷新绘图区，并同步更新右侧结构树。

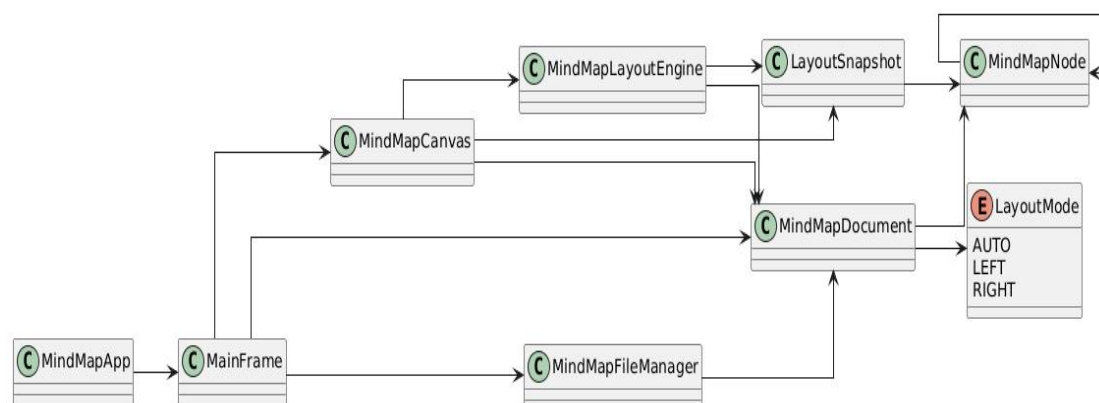
MindMapDocument 表示一整张思维导图，它包含一个中心节点 MindMapNode 和一个布局方式 LayoutMode。中心节点是整棵树的根节点，其他节点都通过子节点列表连接在根节点下方。为了支持撤销操作，MindMapDocument 还提供复制和恢复能力，使系统可以保存操作前的导图快照，并在用户撤销时恢复到之前的状态。

MindMapNode 之间通过父节点引用和子节点列表形成树形结构。普通节点有父节点，也可以有多个子节点；中心节点没有父节点。每个节点除了保存文本内容外，还保存填充颜色、边框颜色、文字颜色、字号、加粗状态、分支颜色以及位置偏移量等信息，用于支持节点样式设置和位置恢复。

MindMapCanvas 在绘制思维导图时，会调用 MindMapLayoutEngine 计算节点位置。MindMapLayoutEngine 根据 MindMapDocument 中保存的结构和 LayoutMode 布局方式生成 LayoutSnapshot。LayoutSnapshot 中保存了每个节点的位置和整张导图的范围，MindMapCanvas 再根据这些结果完成节点和连线绘制。当用户缩放画布、拖动中心节点或导出图片时，MindMapCanvas 也会基于当前布局结果进行显示和输出。

MindMapFileManager 负责把 MindMapDocument 保存为.dt 文件，也负责从.dt 文件中读取数据并恢复为 MindMapDocument 对象。保存时，它会将导图标题、布局方式、节点层级关系、节点样式和位置偏移信息写入 XML；打开文件时，再根据 XML 内容重新构建节点树。这样文件读写逻辑与界面显示逻辑分离，使系统结构更加清晰。

思维导图绘制工具主要类关系图



2.3 数据存储设计

2.3.1 思维导图文件结构设计

本系统使用自定义扩展名 .dt 保存思维导图文件。 .dt 文件主要供本系统保存、读取和继续编辑使用，普通办公软件不能直接以思维导图形式打开。

文件内部采用 XML 格式组织数据。选择 XML 的原因是 XML 本身具有清晰的层级结构，比较适合保存思维导图这种树形数据。系统可以通过 XML 标签的嵌套关系表示节点之间的父子关系，从而较自然地保存整张导图的结构。

.dt 文件主要保存以下内容：

1. 思维导图标题
2. 当前布局方式
3. 中心节点
4. 所有子节点
5. 节点之间的父子关系
6. 节点样式信息
7. 节点位置偏移信息

文件结构示意图如下：

```
<mind-map title="中心主题" layout="AUTO">
  <node id="节点编号"
    text="中心主题"
    fill-color="#E3F1FF"
    border-color="#3874CB"
    text-color="#212B36"
    font-size="14"
    bold="true"
    offset-x="0"
    offset-y="0">
    <node id="节点编号"
      text="子节点 1"
      branch-color="#000000" />
    <node id="节点编号"
      text="子节点 2" />
  </node>
</mind-map>
```

在这个结构中，mind-map 表示整张思维导图，title 表示导图标题，layout 表示当前布局方式。node 表示一个节点，节点之间通过 XML 的嵌套关系表示父子关系。节点的文字、样式和位置偏移等信息则以属性形式保存在对应的 node 标签中。

采用这种结构后，系统在保存文件时可以从中心节点开始递归写入所有子节点；在打开文件时，也可以根据 XML 的嵌套关系递归恢复节点树。

2.3.2 节点数据结构设计

节点是本系统最重要的数据对象。每一个思维导图节点都由 MindMapNode 类表示，整张思维导图则由多个节点按照父子关系组合而成。为了同时支持结构管理、样式设置和位置恢复，节点中不仅保存文字内容，还保存父子关系、显示样式以及位置偏移等数据。

节点数据结构如下表所示：

数据项	数据类型	作用说明
-----	------	------

id	String	节点唯一编号，用于区分不同节点，避免多个节点文字相同时无法准确定位。
text	String	节点显示文字，用于在绘图区和结构树中展示节点名称。
parent	MindMapNode	当前节点的父节点。中心节点没有父节点，普通节点都有父节点。
children	List<MindMapNode>	当前节点的子节点列表，用于保存下级节点并形成树形结构。
fillColorHex	String	节点填充颜色，采用十六进制颜色值保存。
fillTransparent	boolean	是否设置为无填充，用于实现透明背景或仅显示边框的节点效果。
borderColorHex	String	节点边框颜色，采用十六进制颜色值保存。
textColorHex	String	节点文字颜色，采用十六进制颜色值保存。
fontSize	int	节点字体大小，用于控制节点文字显示效果。
bold	boolean	节点文字是否加粗。
branchColorHex	String	当前节点与父节点之间分支线的颜色。
manualOffsetX	int	节点在水平方向上的位置偏移量，用于保存用户调整后的位置。
manualOffsetY	int	节点在垂直方向上的位置偏移量，用于保存用户调整后的位置。

在程序运行过程中，节点之间通过 parent 和 children 建立树形关系。中心节点的 parent 为空，普通节点都有父节点。系统从中心节点开始递归访问所有子节点，就可以遍历整张思维导图。

节点的增删操作也依赖这种树形结构。添加子节点时，系统会创建新的 MindMapNode 对象，并将其加入当前节点的 children 列表，同时设置新节点的 parent；删除节点时，系统会从父节点的 children 列表中移除该节点，该节点下的子节点也会随之从整张导图结构中移除。

2.3.3 节点样式数据设计

为了让思维导图显示得更加清楚，本系统在节点数据中加入了样式信息。相关样式数据保存在 MindMapNode 对象中，并且在保存文件时会一并写入 .dt 文件。这样做的目的是让节点除了保存文字内容和层级关系之外，还能够保留对应的显示效果。

本系统中的节点样式主要包括填充颜色、是否无填充、边框颜色、文字颜色、字体大小、是否加粗以及分支线颜色等内容。其中，填充颜色用于控制节点内部背景颜色；无填充状态用于表示节点是否采用透明背景；边框颜色用于控制节点外边框的显示颜色；文字颜色用于控制节点中文字的颜色；字体大小和是否加粗用于控制文字的显示效果；分支线颜色则用于控制当前节点与父节点之间连接线的颜色。

在保存到文件时，这些样式通常会作为节点属性写入 XML 结构中。例如：

```
<node id="xxx"
  text="系统设计"
  fill-color="#FFFFFF"
  border-color="#72849E"
  text-color="#212B36"
  font-size="14"
  bold="false"
```

```
branch-color="#000000" />
```

采用这种方式后，用户设置好的颜色、字体和分支线效果不会因为关闭程序而丢失。下次重新打开 .dt 文件时，系统可以读取这些样式信息，并将其恢复到绘图区中，从而保持导图前后显示效果的一致性。

2.3.4 布局信息存储设计

本系统支持三种布局方式，分别是自动布局、左侧布局和右侧布局。程序中使用 LayoutMode 枚举类表示这三种布局，对应的取值为 AUTO、LEFT 和 RIGHT。

布局信息保存在 MindMapDocument 对象中。保存文件时，系统会把当前布局方式写入 .dt 文件的根元素中，例如：<mind-map title="中心主题" layout="AUTO">

其中，layout="AUTO"表示当前导图采用自动布局；如果用户选择左侧布局，则保存为 LEFT；如果用户选择右侧布局，则保存为 RIGHT。

AUTO 表示自动布局，在这种模式下，中心节点的一级分支会分布在左右两侧，使整体结构尽量保持平衡；LEFT 表示左侧布局，即中心节点的一级分支主要排列在左侧；RIGHT 表示右侧布局，即中心节点的一级分支主要排列在右侧。

采用这种设计后，布局方式不只是程序运行时的临时状态，而是会作为导图数据的一部分被保存下来。用户下次重新打开 .dt 文件时，系统可以恢复之前使用的布局方式，而不需要每次重新调整。

2.3.5 节点位置偏移信息存储设计

除了自动布局、左侧布局和右侧布局外，系统还支持对中心节点位置进行调整。为了让用户调整后的显示位置能够在保存和重新打开后继续保留，系统在节点数据中加入了位置偏移信息。

位置偏移信息主要包括 manualOffsetX 和 manualOffsetY 两个数据项。其中，manualOffsetX 表示节点在水平方向上的偏移量，manualOffsetY 表示节点在垂直方向上的偏移量。当用户拖动中心节点时，系统会根据鼠标移动距离更新这两个数值，并在重新绘制导图时将偏移量应用到节点位置上。

保存文件时，如果节点存在位置偏移，系统会将偏移量写入对应 node 标签中。例如：

```
<node id="xxx"
  text="中心主题"
  offset-x="60"
  offset-y="-20" />
```

其中，offset-x 表示水平方向偏移量，offset-y 表示垂直方向偏移量。打开文件时，系统会读取这两个属性，并恢复到对应节点对象中。

2.4 界面设计

2.4.1 主界面总体设计

本系统主界面采用常见的桌面软件布局方式。窗口上方为菜单栏和工具栏，中间为主要工作区域，底部为状态栏。整体界面以绘制和编辑思维导图为核心，使用户能够较直观地完成导图的新建、编辑、保存、打开和导出等操作。

中间工作区域分为左右两部分：左侧为绘图区，用于显示和编辑图形化思维导图；右侧为结构展示区，用于以树形结构显示当前导图的节点层级关系。绘图区更适合观察导图整体效果和进行节点编辑，结构展示区更适合查看节点之间的父子关系。当节点数量较多时，用户可以借助结构展示区快速了解导图层次，也可以通过定位中心等操作快速回到中心主题。

主界面主要包括以下区域：

1. 菜单栏
2. 工具栏

3. 绘图区
4. 结构展示区
5. 状态栏
6. 节点右键属性面板

2.4.2 工具栏设计

工具栏位于主界面上方，主要用于放置用户在绘制思维导图时经常使用的功能。

本系统工具栏中包含文件操作、节点编辑、布局切换和画布缩放等功能。文件操作包括新建、打开、保存、另存为和导出。其中，新建用于创建新的思维导图，打开用于读取已有的 .dt 文件，保存用于保存当前导图，另存为用于将当前导图保存为新的文件，导出用于将绘图区内容保存为 PNG 或 JPG 图片。

节点编辑功能包括添加子节点、添加兄弟节点、重命名、取消选中和删除节点。添加子节点用于在当前选中节点下创建新的下级节点；添加兄弟节点用于在当前节点的同一层级新增节点；重命名用于修改当前节点文字；取消选中用于清除当前节点的选中状态；删除节点用于移除当前选中节点及其所有子节点。

工具栏中还提供布局方式选择功能，用户可以在自动布局、左侧布局和右侧布局之间进行切换。布局切换主要用于调整导图整体排布方向，使导图能够适应不同内容和展示场景。

此外，工具栏提供放大、缩小和恢复 100% 显示比例等缩放功能。当导图节点较多或者层级较深时，用户可以通过缩放功能查看整体结构或局部细节。工具栏中的部分按钮会根据当前选中节点状态自动启用或禁用，例如中心节点不能添加兄弟节点，普通节点不能切换布局方式。这样可以减少无效操作，提高界面的易用性。

2.4.3 绘图区设计

绘图区是本系统最核心的操作区域，主要负责显示和编辑思维导图。绘图区中的节点采用圆角矩形显示，节点内部显示文字，父节点与子节点之间使用曲线连接。选中节点时，系统会通过明显的外框效果进行区分，使用户能够清楚地知道当前操作对象。

绘图区需要支持以下操作：

1. 显示中心节点和普通节点
2. 绘制父节点与子节点之间的连线
3. 显示节点选中效果
4. 支持点击节点选中
5. 支持点击空白区域取消选中
6. 支持双击节点重命名
7. 支持右键点击节点弹出属性面板
8. 支持滚动查看较大的导图
9. 支持画布放大、缩小和恢复默认比例
10. 支持 Ctrl 加鼠标滚轮缩放
11. 支持中心节点拖动调整位置
12. 支持将画布内容导出为图片

在绘图区设计中，节点位置不是固定写死的，而是由布局计算部分根据当前节点结构和布局模式动态计算。这样在用户添加、删除节点或切换布局方式后，系统可以自动调整节点位置，使导图整体保持较整齐的状态。

为了改善较大导图的查看体验，绘图区还支持缩放操作。用户既可以通过工具栏按钮进行缩放，也可以使用 Ctrl 加鼠标滚轮在绘图区直接缩放。缩放时，系统会同步处理节点绘制、鼠标命中检测和滚动视图，使用户在放大查看细节或缩小观察整体结构时都能保持较自然的操作体验。

2.4.4 结构展示区设计

结构展示区位于主界面右侧，主要通过树形控件展示当前思维导图的层级关系。中心节点作为树的根节点，其他节点按照父子关系逐级展开显示。与绘图区相比，结构展示区更适合快速查看导图层次，尤其是在节点数量较多时，可以帮助用户更快定位具体内容。

结构展示区上方设置了展开全部、定位中心和清空选择等辅助按钮。展开全部用于显示完整节点层级；定位中心用于将绘图区快速移动到中心节点附近；清空选择用于取消当前节点选中状态。这些操作使结构展示区不仅能显示层级关系，也能辅助用户浏览和演示导图内容。

2.4.5 状态栏设计

状态栏位于主界面底部，主要用于显示当前操作状态和提示信息。虽然状态栏不是主要操作区域，但它可以帮助用户及时了解当前导图的编辑状态，减少操作过程中的不确定性。

本系统状态栏主要显示以下内容：

1. 当前选中的节点名称
2. 当前导图的节点总数
3. 当前文件名称
4. 当前文档是否已修改
5. 操作提示信息
6. 导出后的结果提示

例如，当用户选中某个节点时，状态栏会显示当前节点名称；当导图被修改但尚未保存时，状态栏会提示存在未保存修改；导出成功后，状态栏会显示相应结果。保存成功后，系统会清除未保存修改标记并刷新文件状态。

2.4.6 右键菜单与属性设置界面设计

本系统在绘图区中设计了节点右键属性面板。用户右键点击某个节点后，系统会在节点附近弹出属性面板，使用户可以围绕当前节点直接完成编辑和样式设置，而不需要频繁回到工具栏查找功能。

右键属性面板既包含常用节点操作，也包含节点样式设置。常用操作包括添加子节点、添加兄弟节点和删除节点等。样式设置主要包括以下内容：

1. 节点填充颜色
2. 是否无填充
3. 节点边框颜色
4. 节点文字颜色
5. 字体大小
6. 是否加粗
7. 分支线颜色

节点样式修改后，绘图区会立即刷新显示效果，使用户能够直接看到调整结果。颜色设置通过颜色按钮和调色板进行选择，用户可以快速选择常用颜色，也可以根据需要进行更多颜色。文字样式支持字号和加粗设置，分支线颜色则用于调整节点与父节点之间连接线的显示效果。

2.5 关键设计说明

2.5.1 节点选中机制设计

本系统采用单选机制，也就是同一时间只能有一个节点处于选中状态。用户点击某个节点时，该节点变为选中状态，原先选中的节点取消选中；如果用户点击绘图区空白区域，则取消所有节点的选中状态。

节点被选中后，绘图区会通过高亮边框显示当前选中节点。与此同时，工具栏中的相关

按钮也会根据选中状态进行启用或禁用。例如，没有选中节点时，添加子节点、添加兄弟节点、重命名和删除等操作不应直接执行。

节点选中机制还需要与结构树保持同步。如果用户在绘图区中选中某个节点，右侧结构树中的对应节点也会被同步选中；如果用户在结构树中选中某个节点，绘图区中对应节点也会显示为选中状态。

2.5.2 布局切换设计

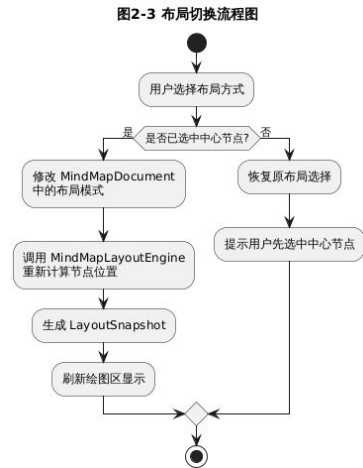
本系统支持自动布局、左侧布局和右侧布局三种方式。布局方式由 LayoutMode 枚举类表示，具体的位置计算由 MindMapLayoutEngine 完成。

在设计上，布局切换不是手动移动某个节点，而是根据当前节点结构重新计算整张导图所有节点的位置。这样可以避免节点位置混乱，也能保证整体排列相对整齐。

自动布局时，系统会把中心节点的一级分支分配到左右两侧，并尽量使左右两侧的整体高度接近；左侧布局时，一级分支主要排列在中心节点左侧；右侧布局时，一级分支主要排列在右侧。

为了避免误操作，本系统要求用户先选中中心节点，再切换布局方式。因为布局是针对整张导图设置的，而不是针对某一个普通节点设置的，这样也更符合用户的理解方式。

布局切换的大致过程如下：



2.5.3 界面同步设计

本系统的界面同步主要包括绘图区、结构展示区、工具栏状态和状态栏信息之间的一致性维护。当节点数据、选中状态或文件状态发生变化时，系统会统一刷新相关界面，避免不同区域显示出不一致的导图状态。

其中，绘图区和结构展示区采用双向同步设计。用户在绘图区选中节点时，结构展示区会定位到对应树节点；用户在结构展示区选中节点时，绘图区也会同步显示该节点的选中效果。工具栏按钮和状态栏则根据当前选中节点、文件状态和文档修改状态动态更新。

2.5.4 撤销机制设计

撤销机制用于在用户误删节点、误改样式或误切换布局后，将导图恢复到上一步编辑状态。由于思维导图同时包含节点结构、节点样式、布局方式和选中状态等数据，撤销操作不能只记录某一个字段的变化，而需要保存一次较完整的导图状态。

本系统采用快照式撤销方案。在添加节点、删除节点、重命名节点、修改节点样式、切换布局以及调整中心节点位置等关键操作执行前，系统会复制当前 MindMapDocument 对象，并将该副本压入撤销栈。快照中保留导图标题、节点层级、节点文字、节点样式、布局方式和位置偏移等信息。

当用户执行撤销操作时，系统从撤销栈中取出最近一次快照，并用该快照恢复当前导图。

这样可以避免分别处理不同操作类型的回退逻辑，使撤销过程更统一，也更适合本系统这种树形结构和样式信息同时变化的场景。

2.5.5 画布缩放与中心定位设计

思维导图在节点数量较少时可以直接完整显示，但当节点数量增多、层级加深后，固定比例显示会影响用户查看整体结构和局部细节。因此，本系统设计了画布缩放与中心定位功能，用于提高大规模导图的浏览和编辑体验。

画布缩放功能主要包括放大、缩小和恢复 100% 显示比例。用户可以通过工具栏按钮调整缩放比例，也可以使用 Ctrl 加鼠标滚轮在绘图区直接缩放。缩放时，系统不仅需要改变绘制比例，还需要同步处理节点点击检测、拖动位移换算和滚动视图位置，避免出现显示位置与鼠标操作位置不一致的问题。

中心定位功能主要用于帮助用户快速回到中心节点位置。当导图较大或用户滚动到较远区域后，用户可以通过结构展示区中的“定位中心”操作，使绘图区自动将中心节点显示到视图中间。

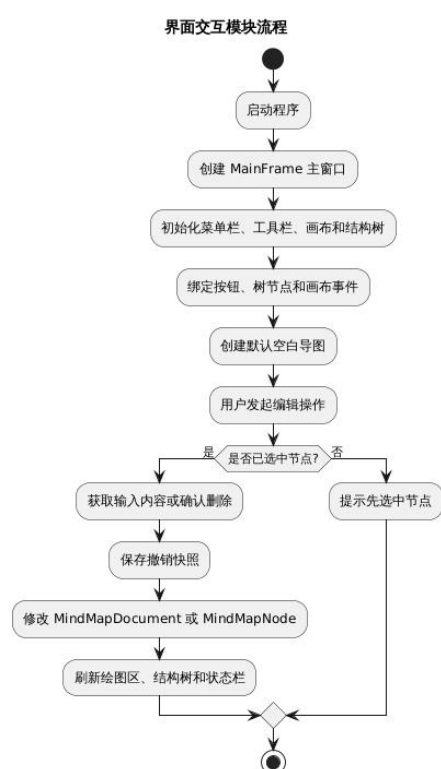
2.5.6 文件覆盖保护设计

在 .dt 文件保存方面，当用户第一次保存导图或执行“另存为”操作时，系统会先弹出文件选择窗口，让用户确定保存路径和文件名。如果用户没有输入 .dt 后缀，系统会自动补全扩展名。确定目标路径后，系统会检查该路径下是否已经存在同名 .dt 文件；如果存在，则弹出覆盖确认提示。只有当用户确认覆盖时，系统才会将当前导图写入该文件；如果用户取消覆盖，则返回保存路径选择流程，原有 .dt 文件不会被修改。

在图片导出方面，用户选择导出路径和图片格式后，系统会根据所选格式自动补全 PNG 或 JPG 扩展名，并检查目标文件是否已经存在。如果不存在同名图片，系统直接执行导出；如果已经存在同名图片，系统会弹出确认提示，询问用户是否覆盖原文件。只有用户确认后，系统才会继续导出；如果用户取消，导出过程会停止，原有图片不会被修改。

3 系统实现

3.1 界面交互模块实现



界面交互模块主要由 MainFrame 类完成，它是整个系统的界面组织中心。程序启动后，MindMapApp 先创建主窗口，再由 MainFrame 初始化菜单栏、工具栏、滚动画布、右侧结构树以及状态栏。这样，系统的所有用户操作都有了统一入口。

在具体实现中，界面交互模块并不是直接修改界面组件，而是采用“先修改数据、再统一刷新界面”的方式。比如添加子节点时，系统先判断当前是否选中了节点，再弹出输入框获取节点名称，然后把新节点加入当前节点的子节点列表，最后调用统一刷新方法更新绘图区、结构树和状态栏。这样做的好处是界面逻辑和数据逻辑不会混在一起，程序结构更清晰。

另外，绘图区与结构树之间还实现了同步选中功能。用户在画布中点击节点时，结构树会定位到对应节点；用户在结构树中选择节点时，画布也会同步高亮对应对象。为了防止两边互相触发形成循环调用，程序中使用了同步标记进行控制。

关键代码片段：

```
public final class MindMapApp {
    private MindMapApp() {
    }

    public static void main(String[] args) {
        try {
            UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
        } catch (Exception ignored) {
        }

        SwingUtilities.invokeLater(() -> {
            MainFrame frame = new MainFrame();
            frame.setVisible(true);
        });
    }
}

private void wireEvents() {
    canvas.setSelectionListener(this::setSelectedNode);
    canvas.setNodeActivationListener(node -> renameSelectedNode());
    canvas.setNodePositionChangeListener(this::handleNodePositionChange);
    canvas.setNodeContextMenuListener((node, point) ->
showCanvasNodePropertyPopup(point));
    bindUndoShortcut();
    bindZoomShortcuts();
    outlineTree.addTreeSelectionListener(event -> {
        if (syncingSelection) {
            return;
        }

        DefaultMutableTreeNode treeNode = (DefaultMutableTreeNode)
outlineTree.getLastSelectedPathComponent();
        MindMapNode node = treeNode == null ? null : (MindMapNode)
treeNode.getUserObject();
        setSelectedNode(node);
    });
    layoutModeBox.addActionListener(event -> {
```

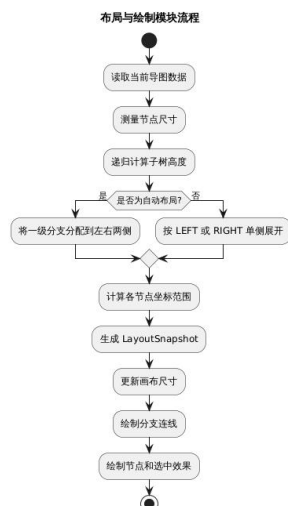
```

        if (syncingLayoutBox || document == null) {
            return;
        }
        LayoutMode layoutMode = (LayoutMode) layoutModeBox.getSelectedItemAt();
        applyLayoutMode(layoutMode);
    });
    addWindowListener(new WindowAdapter() {
        @Override
        public void windowOpened(WindowEvent event) {
            canvas.refreshLayout();
            SwingUtilities.invokeLater(MainFrame.this::focusRootNode);
        }
        @Override
        public void windowClosing(WindowEvent event) {
            closeWindow();
        }
    });
}

private void addChildNode() {
    if (!ensureNodeSelected()) {
        return;
    }
    String text = promptText("请输入子节点名称：", "新节点");
    if (text == null) {
        return;
    }
    pushUndoSnapshot();
    MindMapNode child = selectedNode.addChild(text);
    applyDefaultChildNodeStyle(child);
    markDocumentDirtyAndRefresh(child);
}

```

3.2 布局计算与绘制模块实现



布局与绘制模块由 MindMapLayoutEngine 和 MindMapCanvas 共同完成。MindMapLayoutEngine 负责根据当前节点层级关系和布局方式计算每个节点的位置，MindMapCanvas 负责把节点和分支线真正绘制到画布上。

布局计算时，程序并没有为节点预设固定坐标，而是先递归计算每棵子树的高度，再根据布局方式决定节点向左还是向右展开。自动布局模式下，系统会根据子树高度把中心节点的一级分支分配到左右两侧，尽量使整体结构保持平衡；左侧布局和右侧布局则统一朝单侧展开。计算结果会保存到 LayoutSnapshot 中，后续绘制、点击检测和导出图片都依赖这个快照对象。

绘制时，MindMapCanvas 先根据布局快照刷新画布尺寸，再通过 paintScene() 分阶段完成绘制：先绘制背景，再绘制节点间连线，最后绘制节点本身及选中高亮效果。这样可以避免连线覆盖节点内容，同时也使绘制流程更加清晰。

关键代码片段：

```
public LayoutSnapshot layout(MindMapDocument document, Function<MindMapNode,
Dimension> sizeProvider) {
    Objects.requireNonNull(document, "document");
    Objects.requireNonNull(sizeProvider, "sizeProvider");
    MindMapNode root = document.getRoot();
    Map<String, Integer> subtreeHeights = new HashMap<>();
    computeSubtreeHeight(root, sizeProvider, subtreeHeights);
    Map<String, LayoutSnapshot.NodePlacement> placements = new LinkedHashMap<>();
    Dimension rootSize = sizeProvider.apply(root);
    Rectangle rootBounds = new Rectangle(
        -rootSize.width / 2,
        -rootSize.height / 2,
        rootSize.width,
        rootSize.height
    );
    rootBounds.translate(root.getManualOffsetX(), root.getManualOffsetY());
    placements.put(root.getId(), new LayoutSnapshot.NodePlacement(root,
rootBounds));
    List<MindMapNode> rootChildren = root.getChildren();
    if (rootChildren.isEmpty()) {
        return new LayoutSnapshot(placements);
    }
    if (document.getLayoutMode() == LayoutMode.LEFT) {
        layoutChildGroup(rootBounds, rootChildren, Side.LEFT, sizeProvider,
subtreeHeights, placements);
    } else if (document.getLayoutMode() == LayoutMode.RIGHT) {
        layoutChildGroup(rootBounds, rootChildren, Side.RIGHT, sizeProvider,
subtreeHeights, placements);
    } else {
        RootBuckets buckets = splitRootChildren(rootChildren, subtreeHeights);
        layoutChildGroup(rootBounds, buckets.rightNodes, Side.RIGHT,
sizeProvider, subtreeHeights, placements);
    }
}
```

```

        layoutChildGroup(rootBounds, buckets.leftNodes, Side.LEFT, sizeProvider,
subtreeHeights, placements);
    }
    return new LayoutSnapshot(placements);
}

private void paintScene(Graphics2D graphics, int width, int height, int translateX,
int translateY) {
    graphics.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
    graphics.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING,
RenderingHints.VALUE_TEXT_ANTIALIAS_ON);
    graphics.setColor(Color.WHITE);
    graphics.fillRect(0, 0, width, height);
    if (document == null || snapshot.getPlacements().isEmpty()) {
        return;
    }
    Graphics2D translated = (Graphics2D) graphics.create();
    try {
        translated.scale(zoomScale, zoomScale);
        translated.translate(translateX, translateY);
        drawConnections(translated);
        drawNodes(translated);
    } finally {
        translated.dispose();
    }
}

private void drawConnections(Graphics2D graphics) {
    graphics.setStroke(new BasicStroke(2.0f, BasicStroke.CAP_ROUND,
BasicStroke.JOIN_ROUND));
    for (LayoutSnapshot.NodePlacement placement :
snapshot.getPlacements().values()) {
        MindMapNode node = placement.node();
        if (node.isRoot()) {
            continue;
        }
        LayoutSnapshot.NodePlacement parentPlacement =
snapshot.getPlacement(node.getParent().getId());
        if (parentPlacement == null) {
            continue;
        }
        Rectangle parentBounds = parentPlacement.copyBounds();
        Rectangle childBounds = placement.copyBounds();
        boolean rightSide = childBounds.getCenterX() >=
parentBounds.getCenterX();

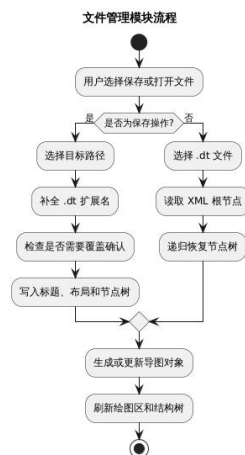
```

```

        int startX = rightSide ? parentBounds.x + parentBounds.width :
parentBounds.x;
        int endX = rightSide ? childBounds.x : childBounds.x + childBounds.width;
        int startY = parentBounds.y + parentBounds.height / 2;
        int endY = childBounds.y + childBounds.height / 2;
        int controlOffset = Math.max(40, Math.abs(endX - startX) / 2);
        CubicCurve2D curve = new CubicCurve2D.Float(
            startX, startY,
            rightSide ? startX + controlOffset : startX - controlOffset,
startY,
            rightSide ? endX - controlOffset : endX + controlOffset, endY,
            endX, endY
        );
        graphics.setColor(withAlpha(resolveBranchColor(node), 200));
        graphics.draw(curve);
    }
}

```

3.3 文件管理模块实现



文件管理模块主要由 MindMapFileManager 类实现，负责思维导图的保存与打开。本系统使用自定义的 .dt 文件格式，该文件本质上是 UTF-8 编码的 XML 文档。之所以采用 XML，是因为 XML 的嵌套结构和思维导图的树形层级关系比较吻合，适合表示节点之间的父子关系。

保存文件时，程序先创建根元素 mind-map，将导图标题和布局方式写入其中，然后从中心节点开始递归写入所有节点。每个节点会保存 id、text、填充颜色、边框颜色、文字颜色、字号、加粗、分支颜色以及手动偏移量等属性。这样，用户重新打开文件时，不仅能恢复节点层级关系，还能恢复节点样式和位置调整结果。

打开文件时，程序先检查 XML 根元素是否符合要求，再读取标题和布局方式，接着递归调用 readNode() 还原节点树，最后构造新的 MindMapDocument 对象并刷新整个界面。通过这种方式，文件持久化逻辑与界面显示逻辑被清楚地分离开来。

关键代码片段：

```

public void save(MindMapDocument document, File file) throws Exception {
    DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
    DocumentBuilder builder = factory.newDocumentBuilder();
}

```

```

    org.w3c.dom.Document xmlDocument = builder.newDocument();
    Element rootElement = xmlDocument.createElement("mind-map");
    rootElement.setAttribute("title", document.getTitle());
    rootElement.setAttribute("layout", document.getLayoutMode().name());
    xmlDocument.appendChild(rootElement);
    rootElement.appendChild(writeNode(xmlDocument, document.getRoot()));
    Transformer transformer = TransformerFactory.newInstance().newTransformer();
    transformer.setOutputProperty(OutputKeys.INDENT, "yes");
    transformer.setOutputProperty(OutputKeys.ENCODING, "UTF-8");
    transformer.setOutputProperty("{http://xml.apache.org/xslt}indent-amount",
    "2");
    transformer.transform(new DOMSource(xmlDocument), new StreamResult(file));
}

private Element writeNode(org.w3c.dom.Document xmlDocument, MindMapNode node) {
    Element element = xmlDocument.createElement("node");
    element.setAttribute("id", node.getId());
    element.setAttribute("text", node.getText());
    writeOptionalAttribute(element, "fill-color", node.getFillColorHex());
    if (node.isFillTransparent()) {
        element.setAttribute("fill-transparent", "true");
    }
    writeOptionalAttribute(element, "border-color", node.getBorderColorHex());
    writeOptionalAttribute(element, "text-color", node.getTextColorHex());
    if (node.getFontSize() != MindMapNode.DEFAULT_FONT_SIZE) {
        element.setAttribute("font-size", String.valueOf(node.getFontSize()));
    }
    if (node.isBold()) {
        element.setAttribute("bold", "true");
    }
    writeOptionalAttribute(element, "branch-color", node.getBranchColorHex());
    if (node.getManualOffsetX() != 0) {
        element.setAttribute("offset-x",
String.valueOf(node.getManualOffsetX()));
    }
    if (node.getManualOffsetY() != 0) {
        element.setAttribute("offset-y",
String.valueOf(node.getManualOffsetY()));
    }
    for (MindMapNode child : node.getChildren()) {
        element.appendChild(writeNode(xmlDocument, child));
    }
    return element;
}

public MindMapDocument load(File file) throws Exception {

```

```

DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
DocumentBuilder builder = factory.newDocumentBuilder();
org.w3c.dom.Document xmlDocument = builder.parse(file);
xmlDocument.getDocumentElement().normalize();
Element rootElement = xmlDocument.getDocumentElement();
if (!"mind-map".equals(rootElement.getNodeName())) {
    throw new ParserConfigurationException("Invalid mind map file.");
}
Element rootNodeElement = firstNodeElement(rootElement);
if (rootNodeElement == null) {
    throw new ParserConfigurationException("Mind map file does not contain any
node.");
}
MindMapNode rootNode = readNode(rootNodeElement);
String title = rootElement.getAttribute("title");
LayoutMode layoutMode =
LayoutMode.fromPersistentName(rootElement.getAttribute("layout"));
return new MindMapDocument(title, rootNode, layoutMode);
}

```

3.4 撤销与状态管理模块实现



撤销与状态管理模块主要负责两件事：一是实现 Ctrl+Z 撤销功能，二是记录当前文档是否存在未保存修改。考虑到本系统中的修改类型很多，不仅包括节点增删改，还包括样式调整、布局切换和中心节点拖动，因此程序没有为每种操作单独设计回退逻辑，而是采用了快照式撤销方案。

所谓快照式撤销，就是在关键操作执行前，先复制一份当前 MindMapDocument 对象，并将其压入撤销栈中。这样，当用户执行撤销时，程序只需取出最近一次快照并恢复文档即可，不必关心之前执行的到底是添加节点、删除节点还是修改样式。这种设计虽然会增加一定内存开销，但实现统一，逻辑更稳定，也更适合树形数据结构。

状态管理方面，系统使用 dirty 变量记录文档是否被修改。只要用户进行了会影响导图

内容的操作，程序就会把 `dirty` 设置为 `true`，并在窗口标题和状态栏中提示“有未保存修改”。当用户尝试新建、打开或关闭窗口时，系统会根据该标志决定是否弹出保存确认框。

关键代码片段：

```
private void pushUndoSnapshot() {
    if (document == null || undoInProgress) {
        return;
    }
    undoStack.push(new UndoSnapshot(
        document.copy(),
        selectedNode == null ? null : selectedNode.getId()
    ));
    while (undoStack.size() > MAX_UNDO_HISTORY) {
        undoStack.removeLast();
    }
    pendingDragUndoNodeId = null;
}

private void undoLastAction() {
    if (document == null) {
        return;
    }
    if (undoStack.isEmpty()) {
        statusLabel.setText("当前没有可撤销的操作。");
        updateWindowState();
        return;
    }
    UndoSnapshot snapshot = undoStack.pop();
    undoInProgress = true;
    try {
        document.restoreFrom(snapshot.documentSnapshot);
        pendingDragUndoNodeId = null;
        dirty = true;
        MindMapNode restoredSelection = snapshot.selectedNodeId == null
            ? document.getRoot()
            : document.findNodeById(snapshot.selectedNodeId);
        refreshDocumentView(restoredSelection == null ? document.getRoot() :
restoredSelection);
    } finally {
        undoInProgress = false;
    }
}

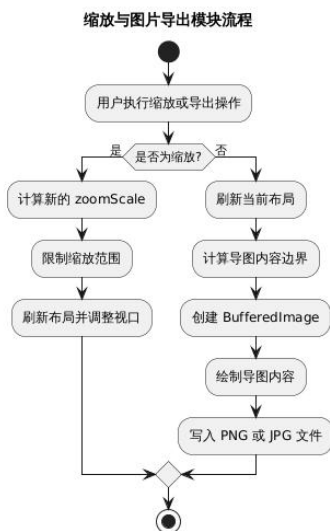
private boolean ensureChangesHandled() {
    if (!dirty) {
        return true;
    }
}
```

```

int choice = JOptionPane.showConfirmDialog(
    this,
    "当前导图有未保存的修改，是否先保存？",
    "未保存修改",
    JOptionPane.YES_NO_CANCEL_OPTION,
    JOptionPane.WARNING_MESSAGE
);
};
if (choice == JOptionPane.CANCEL_OPTION || choice == JOptionPane.CLOSED_OPTION)
{
    return false;
}
if (choice == JOptionPane.YES_OPTION) {
    return saveDocument(false);
}
return true;
}

```

3.5 缩放与图片导出模块实现



缩放与图片导出模块主要由 MindMapCanvas 完成。缩放功能通过 zoomScale 变量控制，用户可以通过工具栏按钮或 Ctrl+鼠标滚轮 来调整当前画布比例。程序会把缩放比例限制在一个合理范围内，避免导图被缩得过小或放得过大。缩放之后，系统会重新计算布局并调整视口，使用户当前关注的位置尽量保持稳定。

图片导出功能复用了当前画布的绘制逻辑。导出时，程序并不是重新单独编写一套绘图代码，而是先刷新布局，得到导图内容的边界范围，再创建对应大小的 BufferedImage，最后直接调用 paintScene() 把导图绘制到图片对象中。这种实现方式的优点是导出效果与界面显示效果一致，而且维护成本较低。

系统支持导出 PNG 和 JPG 两种格式。PNG 使用支持透明通道的图片类型，JPG 使用普通 RGB 图片类型。导出前，主窗口还会检查目标路径是否存在同名文件，并给出覆盖确认提示，以减少误操作带来的影响。

关键代码片段：

```

public boolean setZoomScale(double zoomScale, Point anchorPoint) {

```

```

        double normalizedScale = Math.max(MIN_ZOOM_SCALE, Math.min(MAX_ZOOM_SCALE,
zoomScale));
        if (Math.abs(this.zoomScale - normalizedScale) < 0.0001d) {
            return false;
        }
        Point referencePoint = anchorPoint == null ? null : new Point(anchorPoint);
        Point canvasPointBeforeZoom = referencePoint == null ? null :
toCanvasPoint(referencePoint);
        this.zoomScale = normalizedScale;
        refreshLayout();
        if (referencePoint != null) {
            centerCanvasPointAtView(referencePoint, canvasPointBeforeZoom);
        }
        return true;
    }
    private void handleMouseWheelZoom(MouseWheelEvent event) {
        if (!event.isControlDown()) {
            return;
        }
        int wheelRotation = event.getWheelRotation();
        if (wheelRotation == 0) {
            return;
        }
        if (zoomByStep(wheelRotation < 0 ? 1 : -1, event.getPoint())) {
            event.consume();
        }
    }
    public void exportToImage(File file, String format) throws IOException {
        Objects.requireNonNull(file, "file");
        Objects.requireNonNull(format, "format");
        refreshLayout();
        Rectangle bounds = snapshot.getContentBounds();
        int width = Math.max(1, (int) Math.ceil((bounds.width + CANVAS_MARGIN * 2) *
zoomScale));
        int height = Math.max(1, (int) Math.ceil((bounds.height + CANVAS_MARGIN * 2)
* zoomScale));
        int exportOffsetX = CANVAS_MARGIN - bounds.x;
        int exportOffsetY = CANVAS_MARGIN - bounds.y;
        int imageType = "png".equalsIgnoreCase(format)
            ? BufferedImage.TYPE_INT_ARGB
            : BufferedImage.TYPE_INT_RGB;
        BufferedImage image = new BufferedImage(width, height, imageType);
        Graphics2D graphics = image.createGraphics();
        try {

```

```

        paintScene(graphics, width, height, exportOffsetX, exportOffsetY);
    } finally {
        graphics.dispose();
    }
    ImageIO.write(image, format, file);
}

```

4 系统测试

4.1 测试步骤

本系统测试按照以下步骤进行。

首先，设计测试用例。根据系统各项功能，分别给出测试输入数据、测试操作过程以及预期结果。

其次，实际运行程序。对于文件读写、布局计算和节点样式持久化等核心逻辑，采用 JUnit 单元测试，在项目根目录执行对应测试命令并记录测试结果；对于节点编辑、结构树同步、撤销、缩放、中心定位和图片导出等依赖图形界面的功能，采用手工测试，通过实际操作程序记录运行现象。

最后，分析测试结果。将预期结果与实际运行结果进行比较，判断两者是否一致，并在此基础上得出测试结论。

4.2 模块测试

4.2.1 文件保存与读取模块测试（JUnit）

（1）测试用例设计

输入数据为一张新建的思维导图，将中心节点文本和导图标题设置为“Java 课程设计”，布局方式设置为 LEFT；在中心节点下添加“Swing”和“文件导出”两个一级节点，并在“Swing”节点下继续添加“工具栏”和“绘图区”两个子节点。

预期结果为：导图保存并重新读取后，标题仍为“Java 课程设计”，布局方式仍为 LEFT，节点总数为 5，中心节点下一级子节点数量为 2，且第一个一级子节点的文本内容为“Swing”。

（2）测试指令

在项目根目录下执行以下命令，运行文件保存与读取测试方法：

```
mvn -Dtest=MindMapFileManagerTest#saveAndLoadShouldPreserveStructure test
```

（3）运行结果

实际运行后，测试命令能够正常执行并通过。控制台结果显示 Tests run: 1, Failures: 0, Errors: 0, Skipped: 0。重新读取后的导图标题仍为“Java 课程设计”，布局方式仍为 LEFT，节点总数为 5，中心节点下共有 2 个一级子节点，第一个一级子节点文本为“Swing”。

```

加载个人及系统配置文件用了 3438 毫秒。
(base) PS C:\Users\Administrator\Desktop\java_designwork-czb>
(base) PS C:\Users\Administrator\Desktop\java_designwork-czb> mvn -Dtest=MindMapFileManagerTest#saveAndLoadShouldPreserveStructure test
[INFO] Scanning for projects...
[INFO]
[INFO] --- com.course:mindmap-course-design ---
[INFO] Building Mind Map Course Design 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO]
[INFO] --- jar ---
[INFO]
[INFO] --- resources:3.4.0:resources (default-resources) @ mindmap-course-design ---
[INFO] skip non existing resourceDirectory C:\Users\Administrator\Desktop\java_designwork-czb\src\main\resources
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ mindmap-course-design ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.4.0:testResources (default-testResources) @ mindmap-course-design ---
[INFO] skip non existing resourceDirectory C:\Users\Administrator\Desktop\java_designwork-czb\src\test\resources
[INFO]
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ mindmap-course-design ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ mindmap-course-design ---
[WARNING] UseSystemClassLoader setting has no effect when not forking
[WARNING] The parameter forkCount should likely not be 0. Forking a JVM for tests improves test accuracy. Ensure to have a <forkCount> >= 1.
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnit4PlatformProvider
[INFO] Running com.course.mindmap.MindMapFileManagerTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.583 s -- in com.course.mindmap.MindMapFileManagerTest
[INFO]
[INFO] Results:
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 4.549 s
[INFO] Finished at: 2026-05-13T22:08:47+08:00
[INFO]
(base) PS C:\Users\Administrator\Desktop\java_designwork-czb>

```

(4) 结果分析与测试结论

实际运行结果与预期结果一致，说明文件保存和重新读取后，导图标题、布局方式以及节点层级结构都能够被正确恢复，没有出现数据丢失或结构错误。因此，文件保存与读取模块测试通过。

4.2.2 布局计算模块测试 (JUnit)

(1) 测试用例设计

输入数据为一张空白导图，将布局方式设置为 AUTO，在中心节点下依次添加“需求分析”“界面设计”“文件存储”“图像导出”四个一级子节点。

预期结果为：自动布局计算完成后，中心节点的一级子节点不应全部集中在一侧，而应同时分布在左侧和右侧，从而体现自动布局的平衡效果。

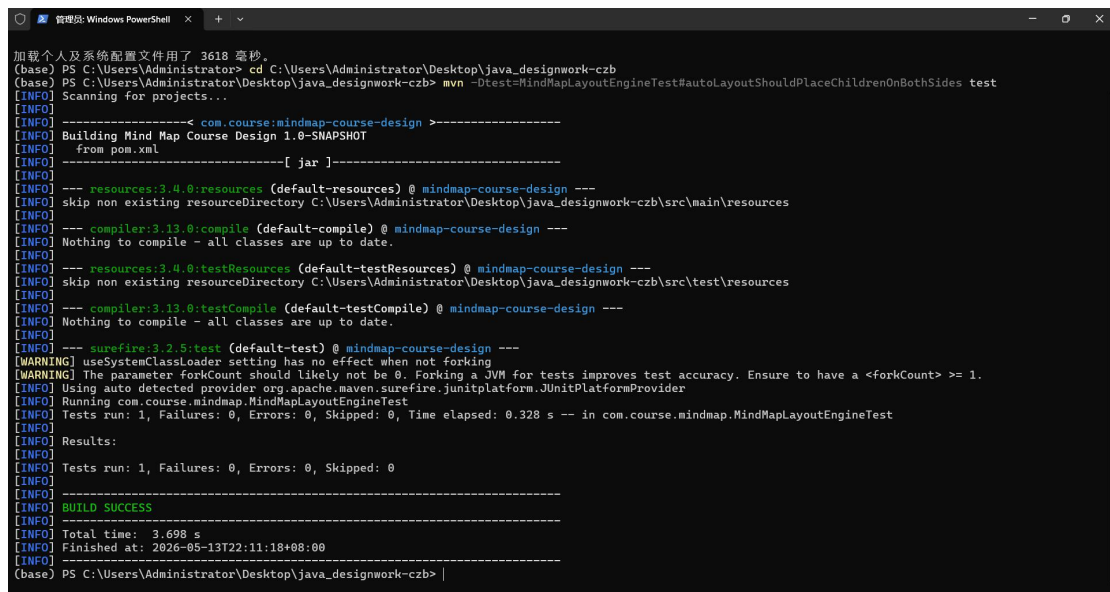
(2) 测试指令

在项目根目录下执行以下命令，运行布局计算测试方法：

```
mvn -Dtest=MindMapLayoutEngineTest#autoLayoutShouldPlaceChildrenOnBothSides test
```

(3) 运行结果

实际运行后，测试命令能够正常执行并通过。控制台结果显示 Tests run: 1, Failures: 0, Errors: 0, Skipped: 0。布局引擎生成的布局快照中包含 5 个节点的位置信息，中心节点左右两侧都存在一级子节点，没有出现全部节点堆积在同一侧的情况。



```
(base) PS C:\Users\Administrator\Desktop\java_designwork-czb> cd C:\Users\Administrator\Desktop\java_designwork-czb
(base) PS C:\Users\Administrator\Desktop\java_designwork-czb> mvn -Dtest=MindMapLayoutEngineTest#autoLayoutShouldPlaceChildrenOnBothSides test
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.course:mindmap-course-design >-----
[INFO] Building Mind Map Course Design 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO]
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.4.0:resources (default-resources) @ mindmap-course-design ---
[INFO] skip non existing resourceDirectory C:\Users\Administrator\Desktop\java_designwork-czb\src\main\resources
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ mindmap-course-design ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.4.0:testResources (default-testResources) @ mindmap-course-design ---
[INFO] skip non existing resourceDirectory C:\Users\Administrator\Desktop\java_designwork-czb\src\test\resources
[INFO]
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ mindmap-course-design ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ mindmap-course-design ---
[WARNING] useSystemClassLoader setting has no effect when not forking
[WARNING] The parameter forkCount should likely not be 0. Forking a JVM for tests improves test accuracy. Ensure to have a <forkCount> >= 1.
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnit4PlatformProvider
[INFO] Running com.course.mindmap.MindMapLayoutEngineTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.328 s -- in com.course.mindmap.MindMapLayoutEngineTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 3.698 s
[INFO] Finished at: 2026-05-13T22:11:18+08:00
[INFO]
(base) PS C:\Users\Administrator\Desktop\java_designwork-czb>
```

(4) 结果分析与测试结论

运行结果表明，自动布局模式下程序能够将一级子节点合理分布到左右两侧，满足预期要求。说明布局计算模块能够依据节点结构生成较为合理的自动布局结果，因此布局计算模块测试通过。

4.2.3 节点样式持久化模块测试 (JUnit)

(1) 测试用例设计

输入数据为一张空白导图。中心节点设置填充颜色为“#445566”、边框颜色为“#778899”、文字颜色为“#112233”、字号为 18、加粗状态为 true；再添加一个名为“Styled Child”的子节点，并设置其为透明填充、边框颜色为“#AA5500”、文字颜色为“#2255AA”、字号为 20、加粗状态为 true、分支颜色为“#00AA88”。

预期结果为：文件保存并重新读取后，中心节点和子节点的填充、边框、文字、字号、加粗和分支颜色等样式信息均与保存前一致。

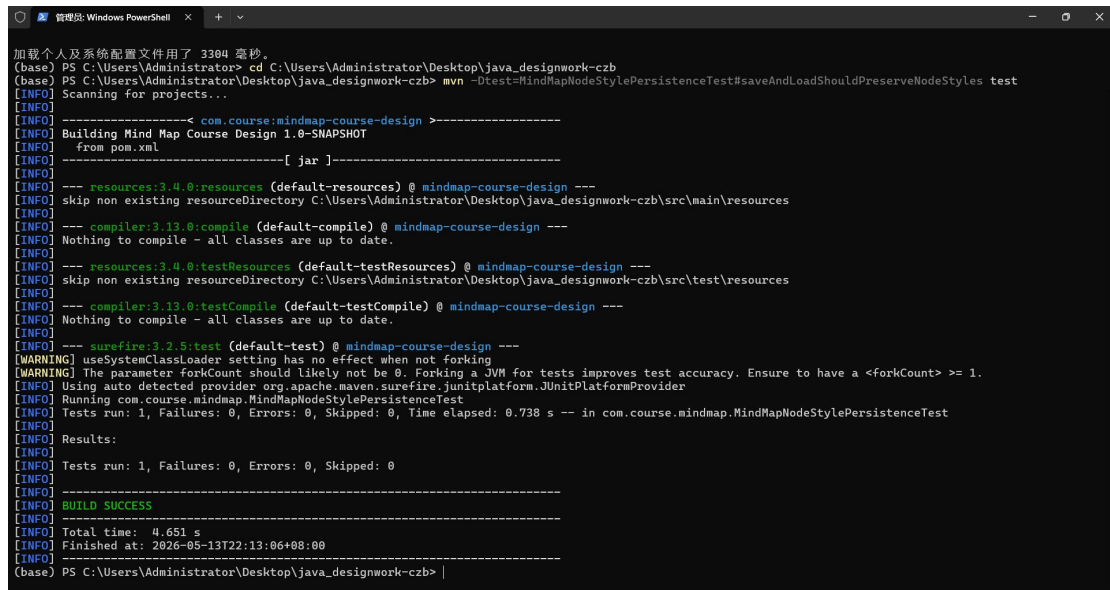
(2) 测试指令

在项目根目录下执行以下命令，运行节点样式持久化测试方法：

```
mvn -Dtest=MindMapNodeStylePersistenceTest#saveAndLoadShouldPreserveNodeStyles test
```

(3) 运行结果

实际运行后，测试命令能够正常执行并通过。控制台结果显示 Tests run: 1, Failures: 0, Errors: 0, Skipped: 0。重新读取后的中心节点仍保持填充颜色“#445566”、边框颜色“#778899”、文字颜色“#112233”、字号 18 和加粗状态；子节点仍保持透明填充、边框颜色“#AA5500”、文字颜色“#2255AA”、字号 20、加粗状态和分支颜色“#00AA88”。



```
(base) PS C:\Users\Administrator\Desktop\java_designwork-czb> mvn -Dtest=MindMapNodeStylePersistenceTest#saveAndLoadShouldPreserveNodeStyles test
[INFO] Scanning for projects...
[INFO] -----< com.course:mindmap-course-design >-----
[INFO] Building Mind Map Course Design 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] --- resources:3.4.0:resources (default-resources) @ mindmap-course-design ---
[INFO] skip non existing resourceDirectory C:\Users\Administrator\Desktop\java_designwork-czb\src\main\resources
[INFO] --- compiler:3.13.0:compile (default-compile) @ mindmap-course-design ---
[INFO] Nothing to compile - all classes are up to date.
[INFO] --- resources:3.4.0:testResources (default-testResources) @ mindmap-course-design ---
[INFO] skip non existing resourceDirectory C:\Users\Administrator\Desktop\java_designwork-czb\src\test\resources
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ mindmap-course-design ---
[INFO] Nothing to compile - all classes are up to date.
[INFO] --- surefire:3.2.5:test (default-test) @ mindmap-course-design ---
[WARNING] useSystemClassLoader setting has no effect when not forking
[WARNING] The parameter forkCount should likely not be 0. Forking a JVM for tests improves test accuracy. Ensure to have a <forkCount> >= 1.
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO] Running com.course.mindmap.MindMapNodeStylePersistenceTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.738 s -- in com.course.mindmap.MindMapNodeStylePersistenceTest
[INFO] Results:
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 4.651 s
[INFO] Finished at: 2026-05-13T22:13:06+08:00
[INFO] (base) PS C:\Users\Administrator\Desktop\java_designwork-czb>
```

(4) 结果分析与测试结论

实际结果与预期结果一致，说明节点样式相关属性已经能够被正确保存到 .dt 文件中，并在重新打开时完整恢复。因此，节点样式持久化模块测试通过。

4.2.4 节点编辑功能测试（手工测试）

(1) 测试用例设计

输入数据为程序启动后自动创建的一张空白导图，中心节点名称为“中心主题”。

测试操作为：先选中中心节点，添加一个子节点“功能模块 A”；再选中“功能模块 A”，添加一个兄弟节点“功能模块 B”；随后将“功能模块 A”重命名为“界面设计”；最后删除“功能模块 B”。

预期结果为：添加子节点后，中心节点下出现新子节点；添加兄弟节点后，两节点位于同一层级；重命名后节点文本立即更新；删除后对应节点从绘图区和结构树中同时消失。

(2) 实际运行程序得到的运行结果

程序运行后，选中中心节点可以正常添加子节点“功能模块 A”；选中“功能模块 A”后可以继续添加兄弟节点“功能模块 B”，两个节点在结构树中处于同一层级；将“功能模块 A”重命名为“界面设计”后，绘图区和结构树中的节点名称都立即更新；删除“功能模块 B”后，该节点在画布和结构树中均被移除。

(3) 结果分析与测试结论

运行结果与预期结果一致，说明节点添加、兄弟节点添加、重命名和删除等编辑功能均能够正常执行，且界面同步更新正常。因此，节点编辑功能测试通过。

4.2.5 中心节点删除限制测试（手工测试）

(1) 测试用例设计

输入数据为程序启动后的默认空白导图，当前选中中心节点。

测试操作为：先观察工具栏“删除节点”按钮的状态，再按下 Delete 快捷键尝试删除中心节点。

预期结果为：当选中中心节点时，工具栏中的“删除节点”按钮应处于不可点击状态；如果用户通过 Delete 快捷键执行删除操作，系统应弹出提示信息，阻止删除中心节点，并保持导图结构不变。

(2) 实际运行程序得到的运行结果

实际运行时，选中中心节点后，工具栏中的“删除节点”按钮处于禁用状态，无法直接点击；按下 Delete 快捷键后，系统弹出提示信息“中心节点不能删除。”。提示出现后，绘图区中的中心节点仍然存在，结构树中的根节点也未发生变化，整张导图结构保持不变。

(3) 结果分析与测试结论

实际运行结果与预期结果一致。程序一方面通过禁用“删除节点”按钮限制用户直接点击删除中心节点，另一方面在用户使用 Delete 快捷键时继续进行提示拦截，形成了双重保护机制，从而避免了根节点被误删而导致导图整体结构被破坏。因此，中心节点删除限制测试通过。

4.2.6 结构树同步显示测试（手工测试）

(1) 测试用例设计

输入数据为一张包含中心节点、“界面设计”和“文件管理”两个一级子节点的导图。

测试操作为：先在绘图区单击“界面设计”节点，再在结构树中单击“文件管理”节点。

预期结果为：在画布中选中节点后，结构树应同步定位并选中对应节点；在结构树中选中节点后，绘图区也应同步高亮对应节点。

(2) 实际运行程序得到的运行结果

实际运行时，在画布中单击“界面设计”节点后，结构树自动选中对应节点；随后在结构树中单击“文件管理”节点时，绘图区中的选中状态同步切换到“文件管理”节点，节点高亮显示正常。

(3) 结果分析与测试结论

运行结果与预期结果一致，说明绘图区和结构树之间已经建立了正确的双向同步机制，因此结构树同步显示测试通过。

4.2.7 撤销功能测试（手工测试）

(1) 测试用例设计

输入数据为一张空白导图。

测试操作为：在中心节点下新增一个子节点“模块测试”，确认该节点已显示后，按下 Ctrl+Z 执行撤销。

预期结果为：撤销后，“模块测试”节点应被移除，导图恢复为添加节点之前的状态。

(2) 实际运行程序得到的运行结果

实际运行时，新增“模块测试”节点后，该节点正常显示在绘图区和结构树中；按下 Ctrl+Z 后，该节点被成功移除，绘图区和结构树同时恢复为仅包含中心节点的状态。

(3) 结果分析与测试结论

运行结果与预期结果一致，说明系统能够在编辑前正确保存快照，并在撤销时恢复导图状态，因此撤销功能测试通过。

4.2.8 画布缩放与中心定位测试（手工测试）

(1) 测试用例设计

输入数据为一张包含多个一级节点和二级节点的导图。

测试操作为：先点击“放大”按钮，再点击“缩小”按钮；随后将滚动视图移动到远离中心节点的位置，最后执行“定位中心”操作。

预期结果为：放大和缩小时导图显示比例应发生变化但内容保持正常；执行定位中心后，视图应重新回到中心节点附近。

（2）实际运行程序得到的运行结果

实际运行时，点击“放大”按钮后，导图显示比例明显增大，节点与连线仍保持正常；点击“缩小”按钮后，比例恢复变小，没有出现显示错位；将视图滚动到较远位置后执行“定位中心”，中心节点重新回到当前视图的主要显示区域。

（3）结果分析与测试结论

运行结果与预期结果一致，说明缩放比例控制、画布刷新以及中心定位功能均工作正常，因此画布缩放与中心定位测试通过。

4.2.9 图片导出模块测试（手工测试）

（1）测试用例设计

输入数据为一张包含多个节点并已设置部分样式的思维导图。

测试操作为：分别执行导出 PNG 和导出 JPG 两次操作，检查目标目录中生成的图片文件；随后再次导出到同一路径，观察系统是否弹出覆盖确认提示。

预期结果为：系统能够正确生成 PNG 和 JPG 图片文件，图片内容完整显示当前导图；若目标路径下已存在同名文件，应弹出覆盖确认提示。

（2）实际运行程序得到的运行结果

实际运行时，导出 PNG 后目标目录成功生成对应图片，打开图片后可完整看到导图内容；导出 JPG 时也能正常生成图片文件，图像显示正常；再次导出到同一路径时，系统弹出覆盖确认对话框，确认后才继续导出。

（3）结果分析与测试结论

运行结果与预期结果一致，说明图片导出功能以及覆盖确认保护功能均能够正常工作，因此图片导出模块测试通过。

4.3 系统测试

4.3.1 系统测试用例设计

输入数据为程序启动后创建的一张空白导图。

测试操作为：依次执行新建导图、添加多个一级和二级节点、修改节点名称、设置节点样式、切换布局方式、保存为 .dt 文件、重新打开该文件、执行一次撤销操作、调整画布缩放比例、定位中心节点，并最终导出 PNG 图片。

预期结果为：在连续执行上述操作的过程中，程序应保持正常运行；节点结构应正确，结构树与绘图区应同步；保存并重新打开后导图内容与样式应保持一致；撤销、缩放、定位和导出功能应能够正常完成。

4.3.2 实际运行程序得到的运行结果

实际运行后，程序从启动到关闭过程均未出现异常。连续添加节点后，节点层级关系保持正确，结构树与绘图区同步更新正常；重命名和样式修改后，界面立即刷新；切换布局方式后，节点分支展开方向发生对应变化；保存为 .dt 文件后重新打开，导图标题、布局方式、节点结构和节点样式均能恢复；执行 Ctrl+Z 后，最近一次编辑操作能够被撤销；缩放和中心定位功能运行正常；导出 PNG 后成功生成图片文件，图片内容与当前导图一致。

4.3.3 结果分析与测试结论

系统测试中，实际运行结果与预期结果一致，没有出现节点丢失、层级错误、界面不同步、文件恢复失败或导出异常等问题。这说明各功能模块在组合使用时能够保持稳定协同工作，系统整体运行正常，能够满足思维导图绘制工具的基本使用要求。

4.4 测试结论

通过以上 JUnit 单元测试和手工测试可以看出，本系统的主要功能均能够正常运行。Junit 测试部分共执行了 3 个测试，实际运行结果为 Tests run: 3, Failures: 0, Errors: 0, Skipped: 0，说明文件保存与读取、自动布局以及节点样式持久化等核心逻辑均已通过验证；手工测试部分也表明节点编辑、结构树同步、撤销、缩放、中心定位和图片导出等界面功能运行正常。综合来看，本系统整体实现较为完整，运行较为稳定。

5 系统运行界面

本章主要展示系统的实际运行效果，通过主界面、节点编辑、样式设置、布局切换、文件操作、画布缩放、中心定位和覆盖确认等截图，直观呈现系统主要功能的界面表现。

5.1 主界面截图

图 5-1 展示了系统启动后的主界面。界面上方为菜单栏和工具栏，中间为思维导图绘图区，右侧为结构显示区，底部为状态栏。截图可以看出，系统启动后会直接显示中心节点，用户可以在该界面中继续完成导图编辑。

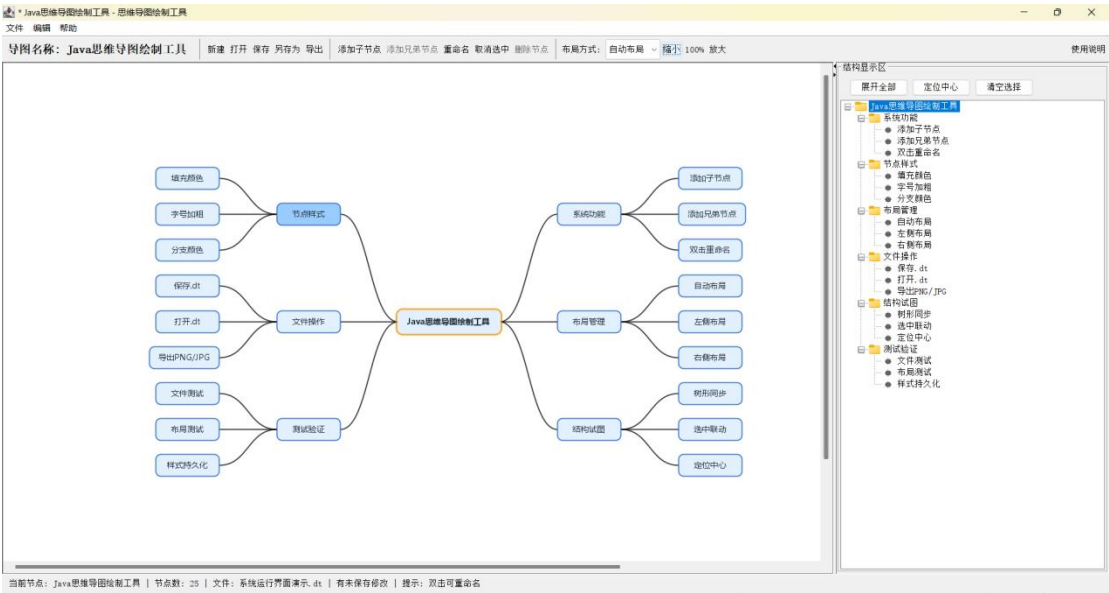


图 5-1 系统主界面

5.2 节点编辑功能截图

图 5-2 展示了节点编辑后的界面效果。截图中可以看到，新增节点会显示在绘图区中，右侧结构显示区也会呈现对应的层级关系，便于观察节点之间的从属结构。

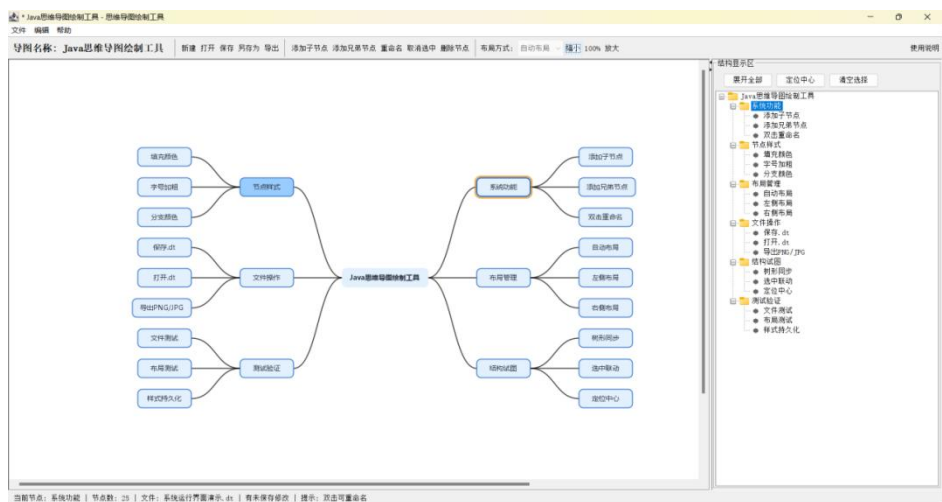


图 5-2 节点编辑功能效果图

5.3 节点样式设置功能截图

图 5-3 展示了节点样式设置界面及设置后的显示效果。截图中可以看到，节点颜色、文字样式和分支线样式能够在绘图区中直观呈现，不同节点之间的视觉区分更加明显。

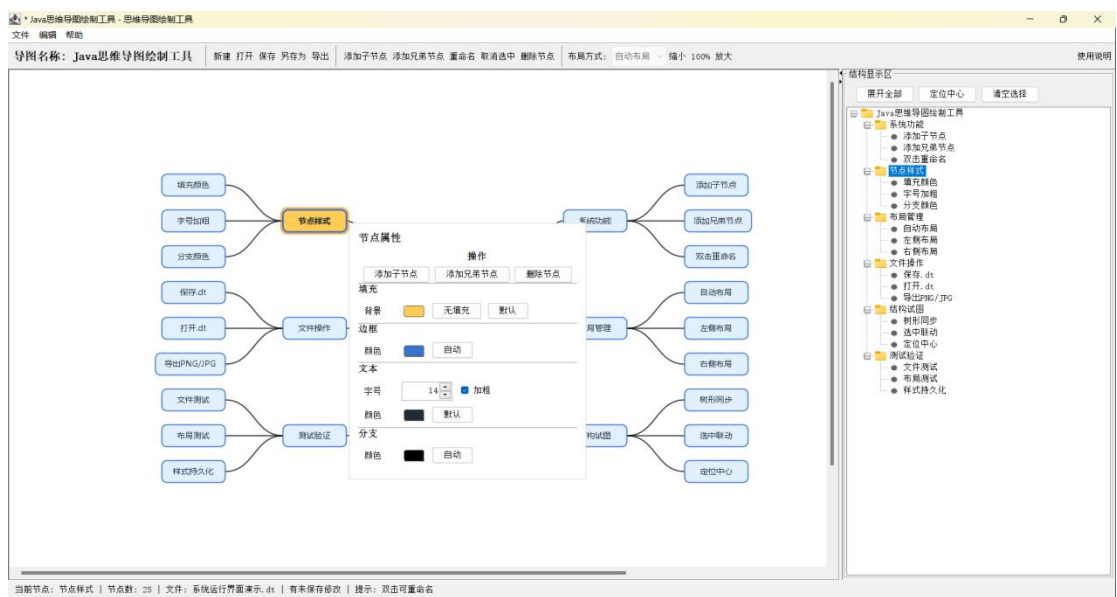


图 5-3 节点样式设置功能截图

5.4 布局切换效果截图

图 5-4 展示了自动布局、左侧布局和右侧布局三种显示效果。不同截图中，节点层级关系保持一致，但分支在画布中的展开方向和整体排布位置发生变化，便于对比不同布局模式的显示差异。

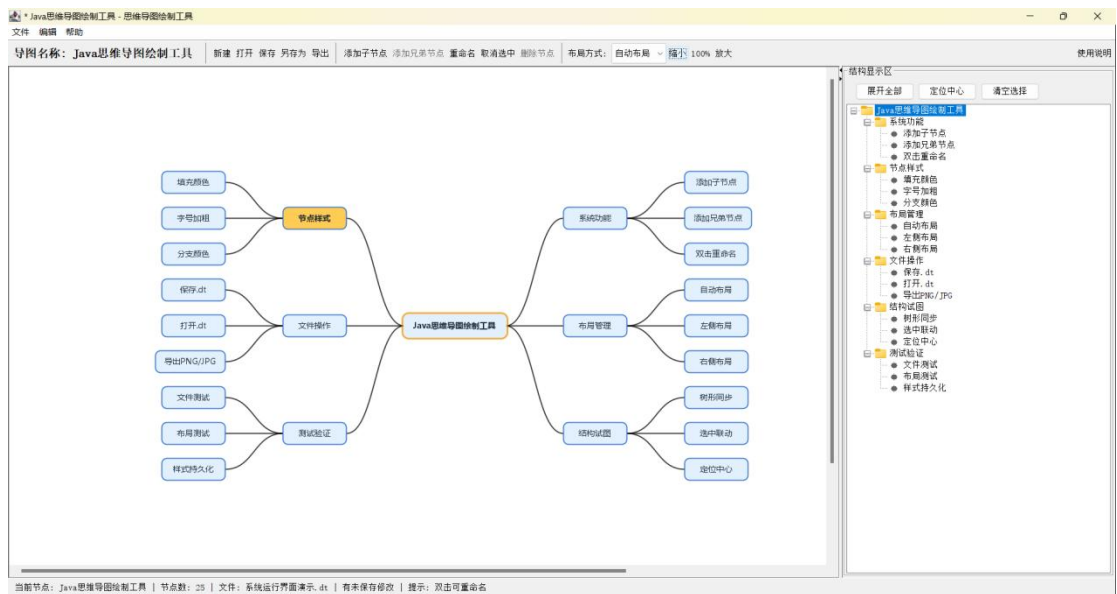


图 5-4 (a) 自动布局效果图

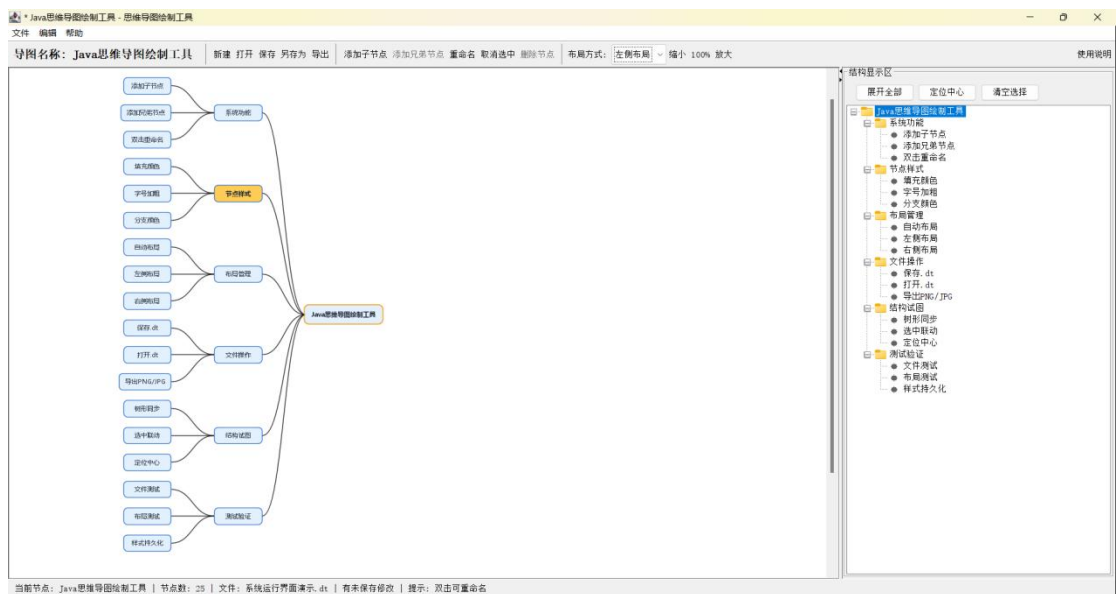


图 5-4 (b) 左侧布局效果图

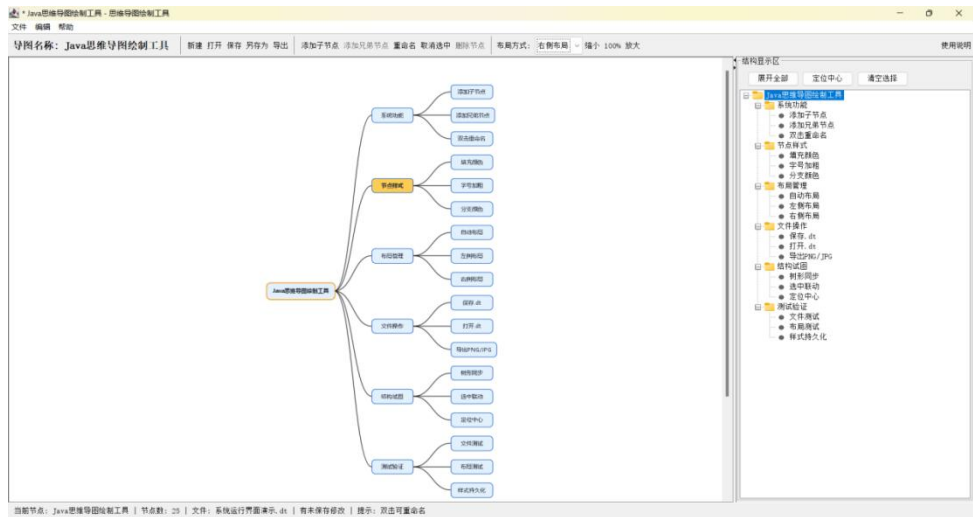


图 5-4(c) 右侧布局效果图

5.5 文件保存、打开与导出效果截图

图 5-5 展示了文件保存、重新打开以及图片导出的运行效果。截图中可以看到，导图内容可以保存为本地文件，也可以导出为图片文件，用于后续编辑或成果展示。

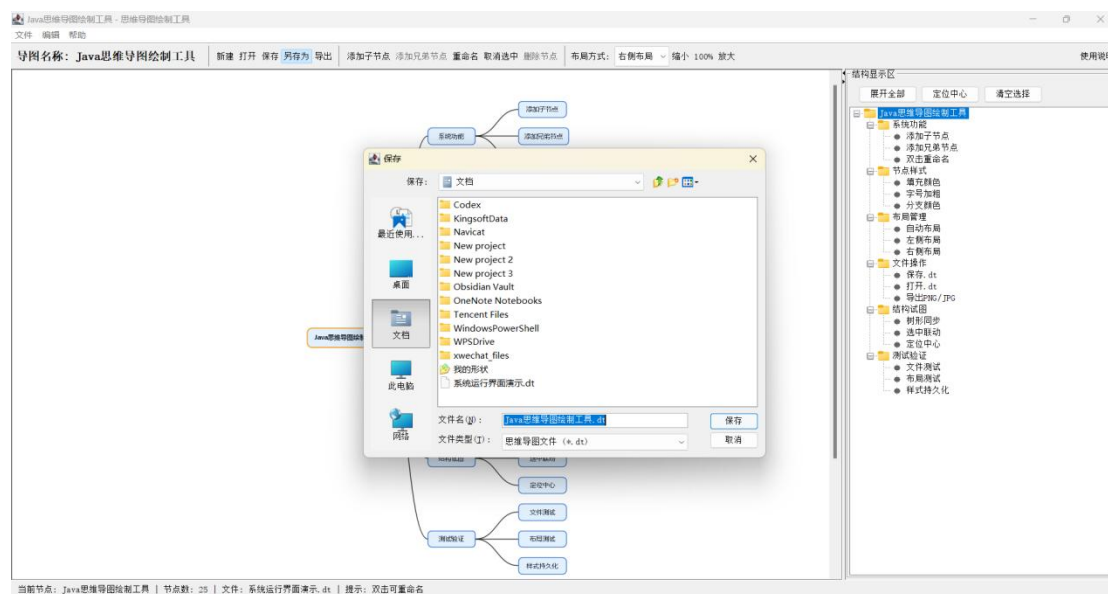


图 5-5(a) 文件保存效果图

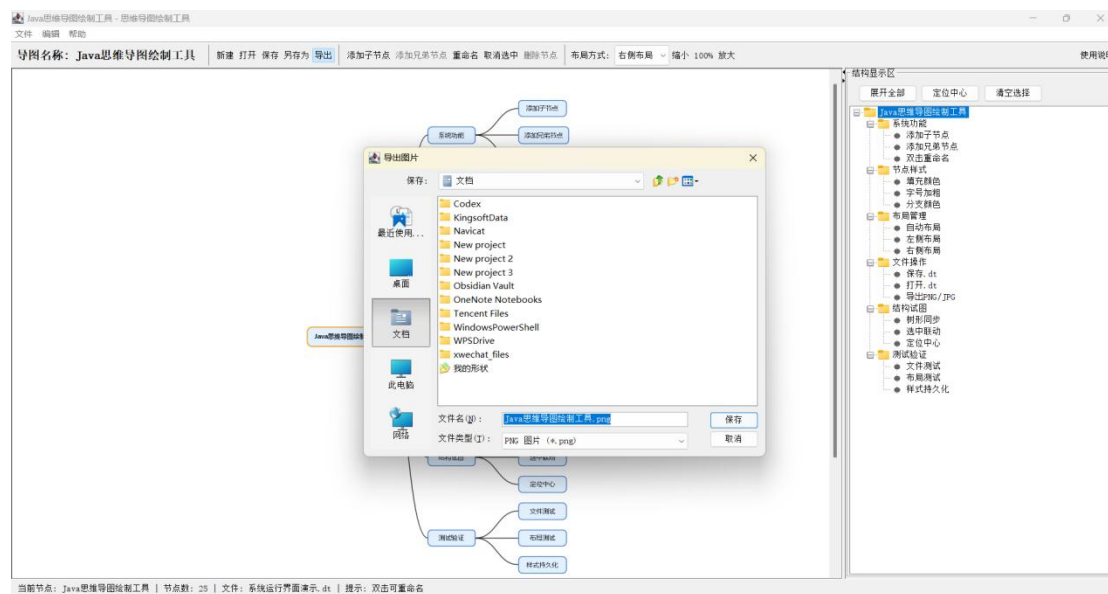


图 5-5(b) 文件导出效果图

5.6 画布缩放与中心定位效果截图

图 5-6 展示了画布缩放和中心定位后的界面效果。通过缩放截图可以观察导图显示比例变化后的效果；通过中心定位截图可以看到视图回到中心节点附近后的显示状态。

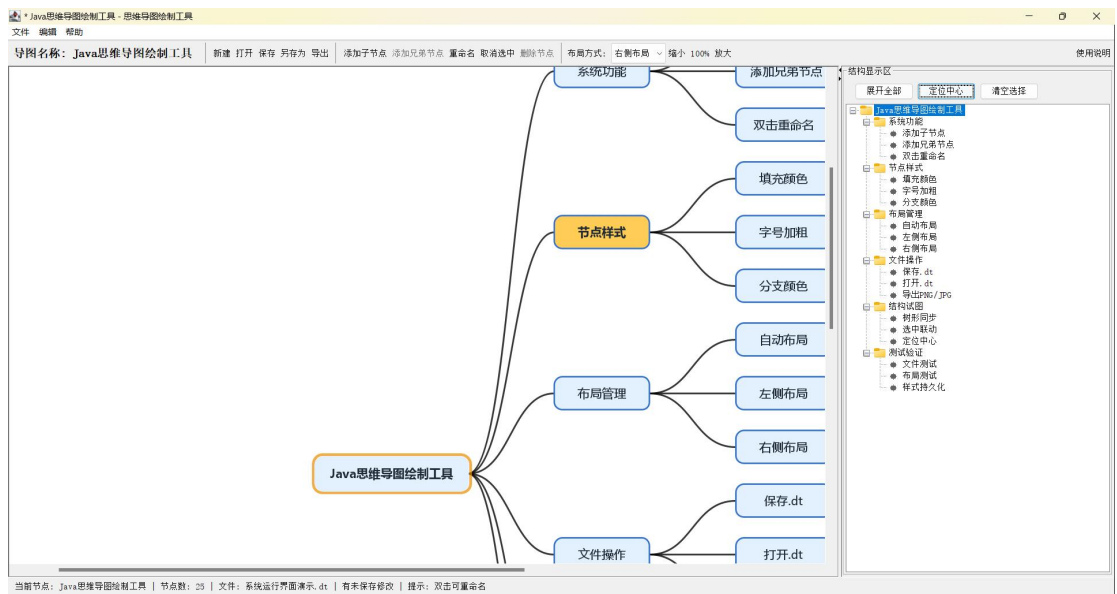


图 5-6(a) 画布缩放效果图

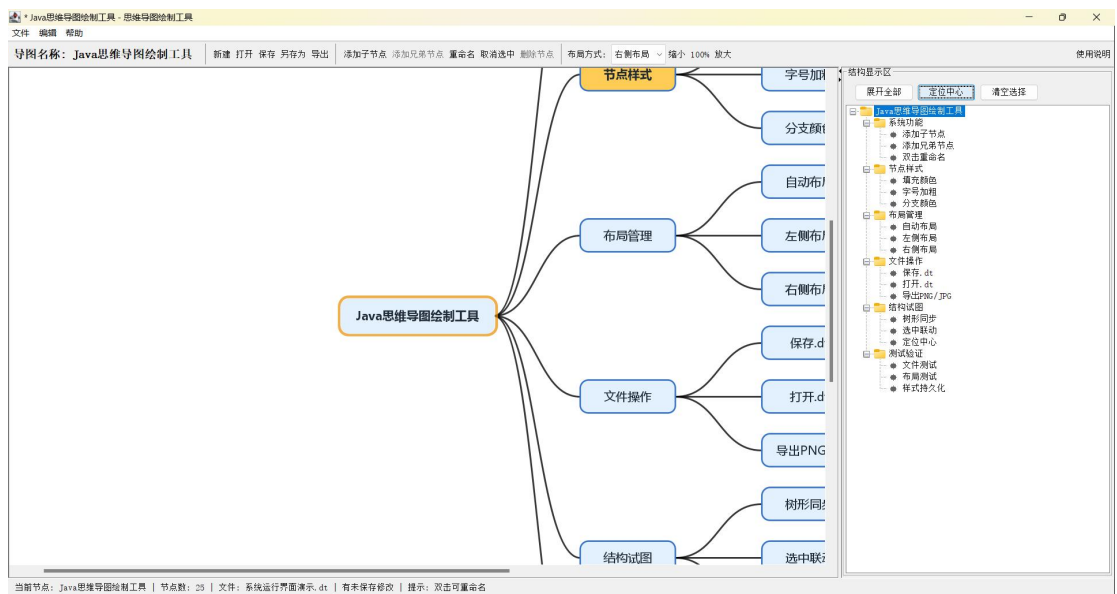


图 5-6(b) 中心节点定位效果图

5.7 撤销截图

图 5-7 展示了撤销操作前后的界面变化。截图可以看出，撤销后导图状态会回到上一步操作前。

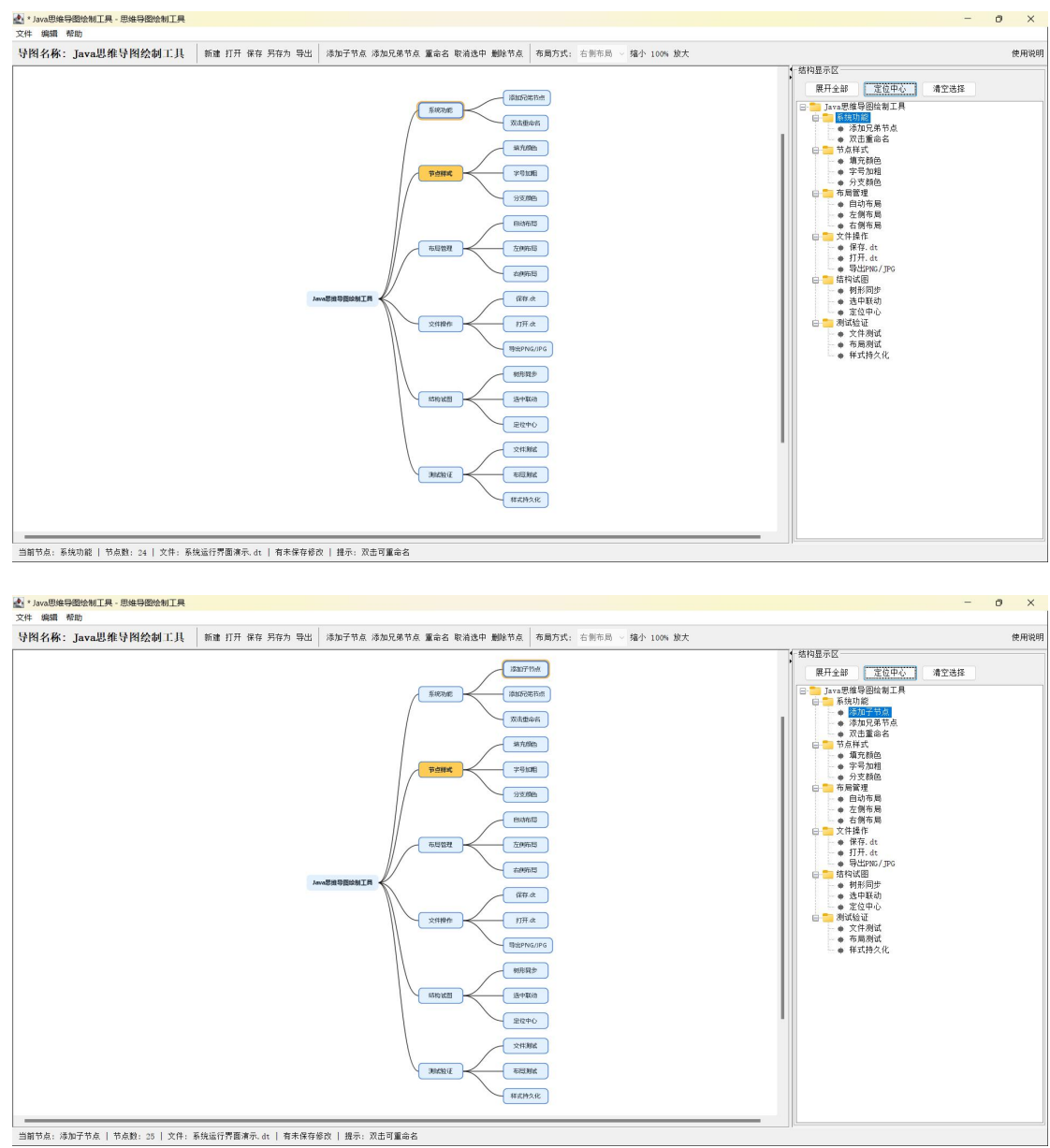


图 5-7 撤销操作前后效果图

5.8 文件覆盖确认效果截图

图 5-8 展示了文件或图片导出后的覆盖提示效果。截图可以看出，当导出图片或保存到已存在的同名文件路径时，系统会显示覆盖确认提示。

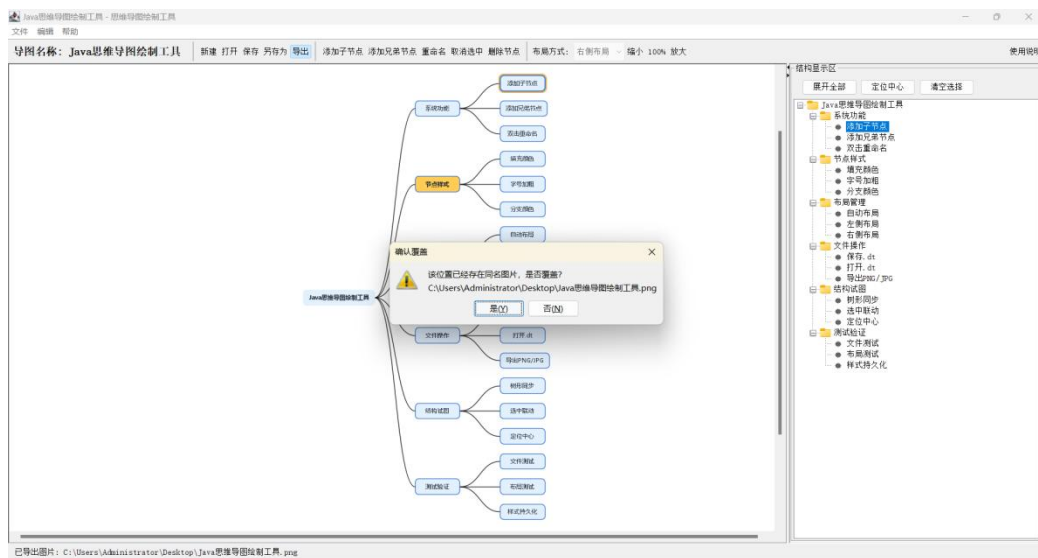


图 5-8(a) 导出覆盖确认提示图

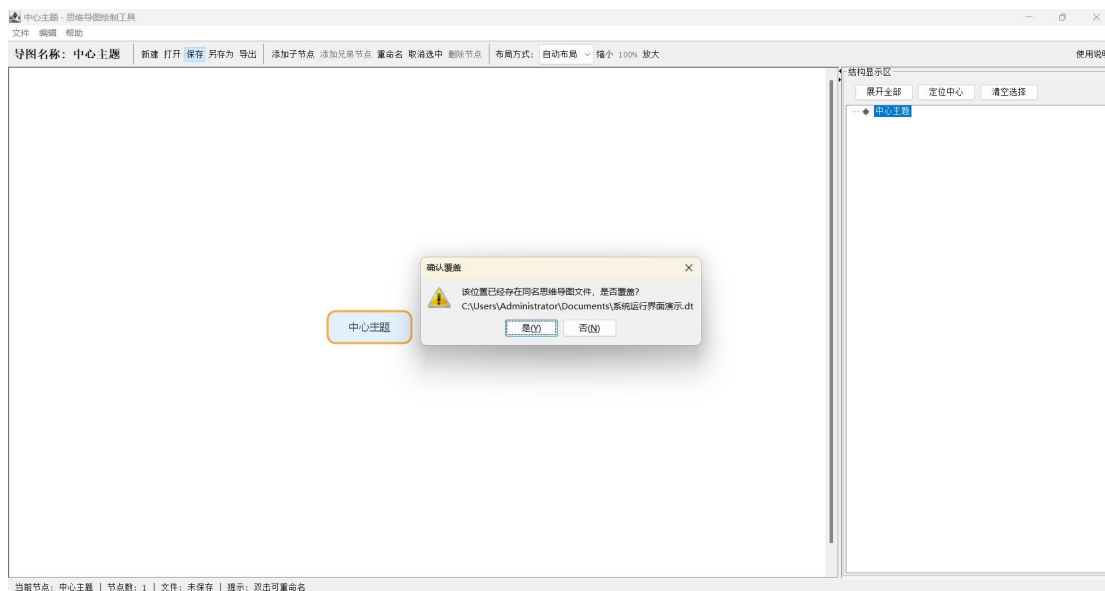


图 5-8(b) 文件覆盖确认提示图

6 总结

(1) 陈宗波

在本次课程设计中，我主要担任组长，负责项目整体进度推进、任务协调和阶段性验收。在开发过程中，我重点完成了需求分析、总体设计以及核心数据模型设计，并负责文件管理模块的实现，包括文件的新建、打开、保存以及自定义格式的设计与解析。同时，我还承担了成员代码整合、版本管理、联调测试和最终打包工作，并参与答辩材料统筹和论文整体框架整理。

通过这次项目实践，我更加深刻地体会到团队协作和项目管理的重要性。作为组长，不仅要关注自己负责的功能实现，还要及时协调成员之间的工作衔接，保证整体进度和质量。这次经历提升了我的系统设计能力、代码整合能力以及统筹规划能力，也让我认识到在实际开发中，清晰的前期设计和良好的沟通机制对项目成功具有重要意义。

(B) 温文辉

在本次课程设计中，我主要负责系统主界面的搭建与交互功能实现，包括工具栏、绘图区和结构展示区的界面布局设计。在功能实现方面，我完成了节点选中、取消选中、按钮事件绑定等交互逻辑，并负责树形结构展示区与绘图区之间的同步更新。此外，我还参与了界面美化、提示信息完善以及演示流程优化等工作，并负责论文中“系统实现”“系统运行界面”部分的初稿整理。

通过本次开发，我对 Java 图形界面设计和事件驱动编程有了更加深入的理解。尤其是在界面布局和交互细节处理过程中，我体会到用户体验对软件质量的重要影响。一个界面不仅要功能完整，还需要操作直观、反馈清晰。这次实践提高了我的界面设计能力和前端交互实现能力，也让我学会了如何从用户角度思考问题、不断优化程序细节。

(3)陈宇冲

在本次课程设计中，我主要负责布局算法设计与实现，包括自动布局、左侧布局和右侧布局等功能；同时完成了节点添加、删除逻辑以及节点连线绘制功能。在此基础上，我还负责导出模块的开发，实现了将绘图区内容导出为 PNG、JPG 等图像文件。此外，我参与了系统功能测试、测试用例整理和缺陷记录工作，并负责论文中“系统测试”“总结”部分的初稿整理。

通过本次项目，我对算法设计、图形绘制以及软件测试流程有了更全面的认识。布局算法的实现让我体会到，功能不仅要“能用”，还要尽量做到结构清晰、效果合理；而测试工作则让我认识到，发现问题、记录问题和改进问题同样是软件开发中不可缺少的重要环节。这次课程设计锻炼了我的编程实现能力、问题分析能力和测试思维，也让我对完整的软件开发流程有了更真实的理解。